# Task 1: Create a Customized Docker Image Using a Dockerfile

## 1. Objective

To create a customized Docker image using a Dockerfile, run the image as a container on an AWS EC2 Ubuntu instance, and verify that the application is accessible through a web browser.

## 2. Why do we need to perform this task?

In real-world DevOps projects, applications need to be packaged and deployed consistently across different environments, such as development, testing, and production.

A Dockerfile allows us to define how an image should be created, including the base image, application files, and startup command.

Why this is useful:

- Consistency: The same image can run across different environments.
- Automation: We can build images using repeatable instructions instead of configuring each server manually.
- Portability: Docker images can be pushed to registries such as Docker Hub and Amazon ECR and deployed on other servers.
- Customization: We can package our own application content instead of using only the default Nginx webpage.
- DevOps integration: The Dockerfile can later be used in a Jenkins CI/CD pipeline to automate image building and deployment.

## 3. Steps Used

### Step 1: Create a project directory

```
mkdir -p ~/docker-challenge/task-01
cd ~/docker-challenge/task-01
```

Created a separate directory to store the Dockerfile and application files.

### Step 2: Create the customized webpage

Created `index.html` containing the DevOps stages and tools, including Git, GitHub, Jenkins, Maven, SonarQube, Docker, Terraform, AWS, and monitoring tools.

```
nano index.html
```

### Step 3: Create the Dockerfile

Created a file named `Dockerfile` with these instructions:

```
FROM nginx:latest

LABEL maintainer="Asifshaik"

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

- `FROM`: Uses the official Nginx image as the base.
- `LABEL`: Adds maintainer information.
- `COPY`: Copies the customized webpage into the Nginx web directory.
- `EXPOSE`: Documents the port on which Nginx listens inside the container.
- `CMD`: Starts Nginx in the foreground.

### Step 4: Build the customized Docker image

```
sudo docker build -t asif-custom-nginx:v1 .
```

Created a Docker image named `asif-custom-nginx` with the tag `v1`.

### Step 5: Remove the previous container

During execution, an existing container used the name `challenge-nginx`. Removed it before recreating the container:

```
sudo docker rm -f challenge-nginx
```

This removes the container, not the Docker image or the project files.

### Step 6: Run the Docker container

```
sudo docker run -d \
  --name challenge-nginx \
  -p 8090:80 \
  asif-custom-nginx:v1
```

Started the container in detached mode and mapped EC2 host port `8090` to container port `80`.

### Step 7: Verify the container and webpage

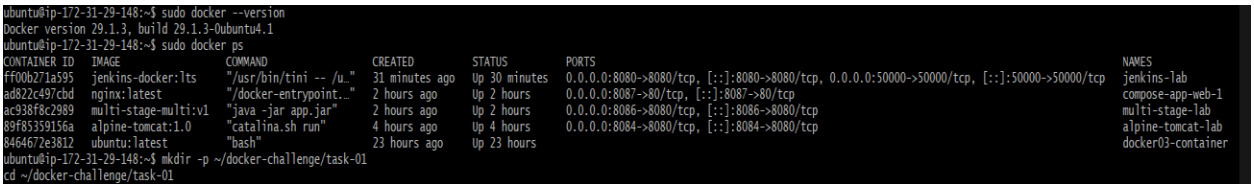
Checked the running container:

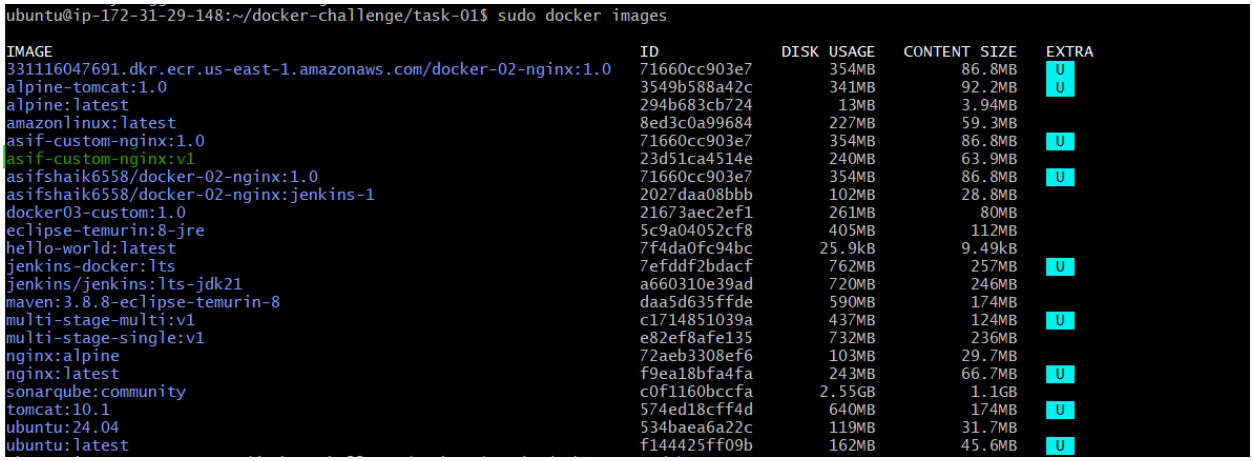
```
sudo docker ps
```

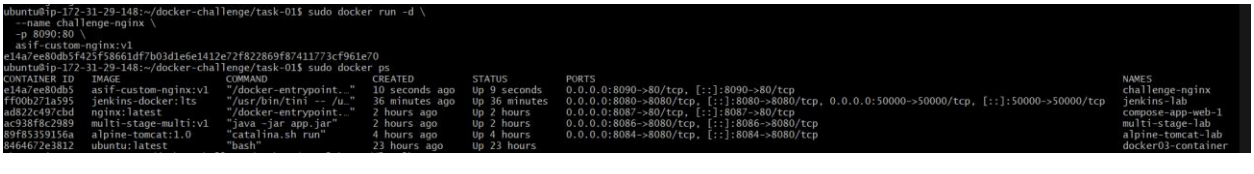
Tested the webpage from the EC2 instance:

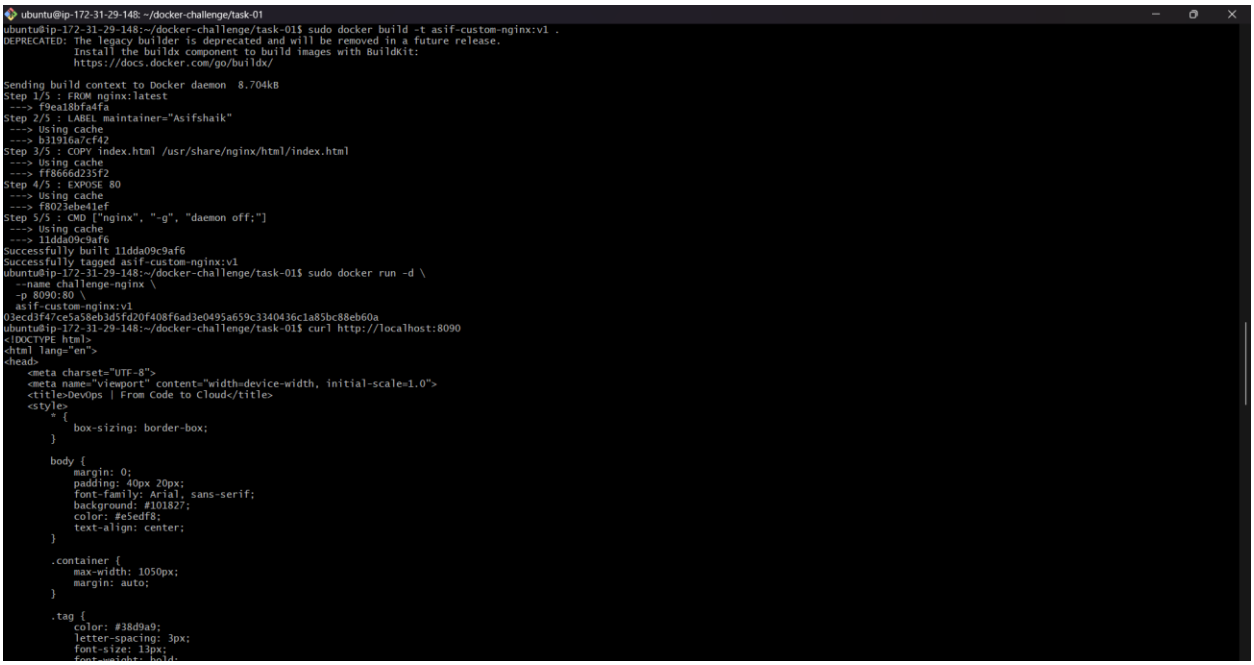
```
curl http://localhost:8090
```

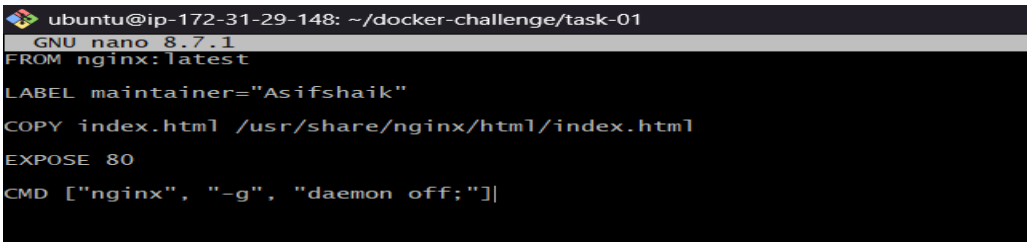
The customized webpage was displayed successfully.

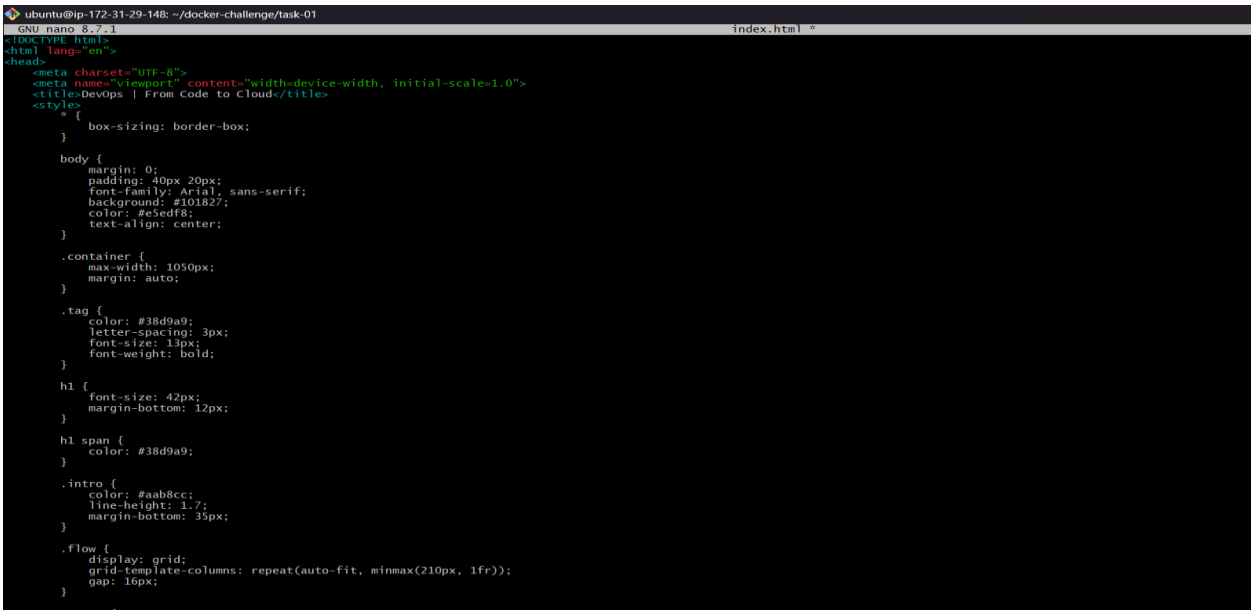


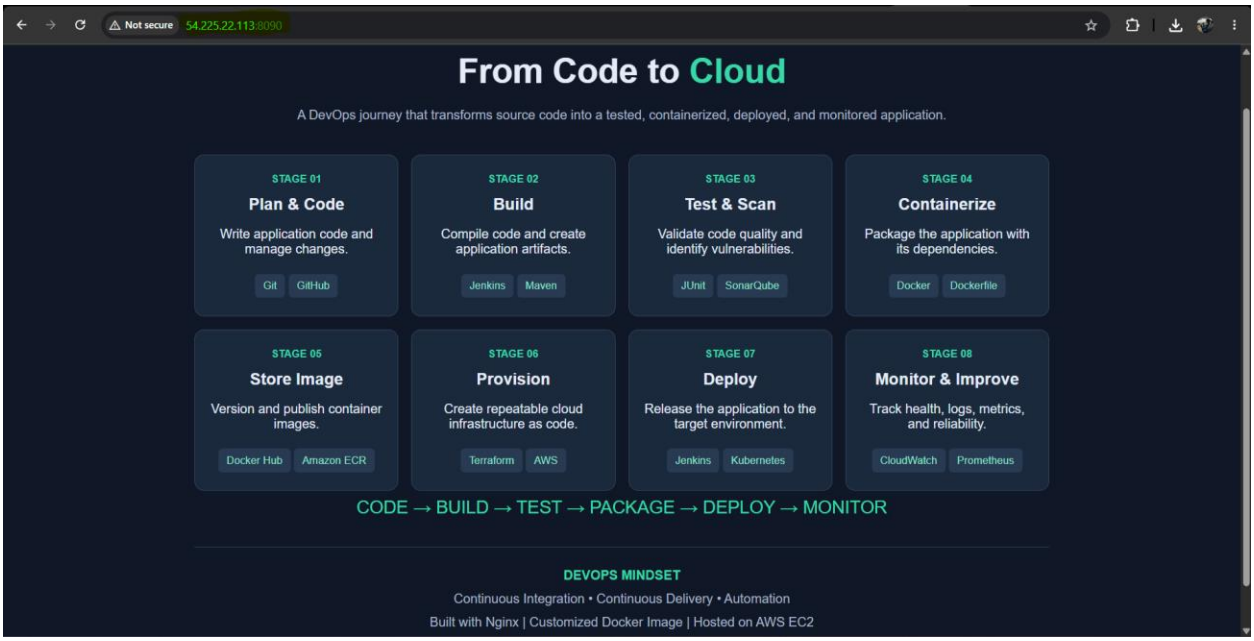




ubuntu@ip-172-31-29-148: ~/docker-challenge/task-01
ubuntu@ip-172-31-29-148:~/docker-challenge/task-01$ sudo docker build -t asif-custom-nginx:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
Sending build context to Docker daemon 8.704kB
Step 1/5 : FROM nginx:latest
---> f9ea18bfa4fa
Step 2/5 : LABEL maintainer="Asifshaik"
---> Using cache
---> b31916a7cf42
Step 3/5 : COPY index.html /usr/share/nginx/html/index.html
---> Using cache
---> ff8666d235f2
Step 4/5 : EXPOSE 80
---> Using cache
---> f8023ebe41ef
Step 5/5 : CMD ["nginx", "-g", "daemon off;"]
---> Using cache
---> 11dda09c9af6
Successfully built 11dda09c9af6
Successfully tagged asif-custom-nginx:v1
ubuntu@ip-172-31-29-148:~/docker-challenge/task-01$ sudo docker run -d \
--name challenge-nginx \
-p 8090:80 \
asif-custom-nginx:v1
03ecd3f47ce5a58eb3d5fd20f408f6ad3e0495a659c3340436c1a85bc88eb60a
ubuntu@ip-172-31-29-148:~/docker-challenge/task-01$ curl http://localhost:8090
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>DevOps | From Code to Cloud</title>
<style>
* {
box-sizing: border-box;
}
body {
margin: 0;
padding: 40px 20px;
font-family: Arial, sans-serif;
background: #101827;
color: #e5edf8;
text-align: center;
}
.container {
max-width: 1050px;
margin: auto;
}
.tag {
color: #38d9a9;
letter-spacing: 3px;
font-size: 13px;


ubuntu@ip-172-31-29-148: ~/docker-challenge/task-01
GNU nano 8.7.1
FROM nginx:latest
LABEL maintainer="Asifshaik"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]


ubuntu@ip-172-31-29-148: ~/docker-challenge/task-01
GNU nano 8.7.1 index.html *
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>DevOps | From Code to Cloud</title>
<style>
* {
box-sizing: border-box;
}
body {
margin: 0;
padding: 40px 20px;
font-family: Arial, sans-serif;
background: #101827;
color: #e5edf8;
text-align: center;
}
.container {
max-width: 1050px;
margin: auto;
}
.tag {
color: #38d9a9;
letter-spacing: 3px;
font-size: 13px;
font-weight: bold;
}
h1 {
font-size: 42px;
margin-bottom: 12px;
}
h1 span {
color: #38d9a9;
}
.intro {
color: #aab8cc;
line-height: 1.7;
margin-bottom: 35px;
}
.flow {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(210px, 1fr));
gap: 16px;
}




## 4. Observation

- The custom Docker image was built successfully using the Dockerfile.
- The image was created with the name `asif-custom-nginx:v1`.
- The container `challenge-nginx` ran successfully on port `8090`.
- The customized DevOps webpage was accessible through the local curl request.
- Nginx served the webpage from inside the Docker container.

## 5. Conclusion

Successfully created a customized Docker image using a Dockerfile and deployed it as a running Nginx container on an AWS EC2 Ubuntu instance. Verified that the customized DevOps webpage was served successfully on port `8090`.

This task demonstrated how Dockerfiles can be used to create repeatable, portable application images that can later be pushed to Docker Hub and Amazon ECR and integrated into a Jenkins CI/CD pipeline.

# Task 2: Push a Customized Docker Image to Docker Hub

## 1. Objective

To tag and push the customized Docker image `asif-custom-nginx:v1` from an AWS EC2 Ubuntu instance to Docker Hub, making the image available in a remote container registry.

## 2. Why do we need to perform this task?

Building a Docker image stores it locally on the machine where it was created. Other servers cannot directly access that local image.

Docker Hub provides a remote registry where we can store and share Docker images.

Purpose:

- Image sharing: Make the image accessible to other developers and servers.
- Portability: Pull the image onto another machine without rebuilding it.
- Deployment: Use the same image in testing and production environments.
- CI/CD integration: Allow Jenkins pipelines to automatically push newly built images to Docker Hub.
- Version management: Use tags such as `v1`, `v2`, and `v3` to identify image versions.

## 3. Steps Used

### Step 1: Verify the local Docker image

Command:

```
sudo docker images
```

Verified that the customized image `asif-custom-nginx` with tag `v1` was available locally.

### Step 2: Log in to Docker Hub

Command:

```
sudo docker login
```

Authenticated with Docker Hub so the EC2 instance could push images to the registry.

### Step 3: Tag the Docker image

Command:

```
sudo docker tag asif-custom-nginx:v1 YOUR_DOCKERHUB_USERNAME/asif-custom-nginx:v1
```

Replaced `YOUR_DOCKERHUB_USERNAME` with the actual Docker Hub username.

Tagging associates the local image with the destination repository name and version tag.

### Step 4: Verify the tagged image

Command:

```
sudo docker images
```

Verified that the image appeared under the Docker Hub repository name with the `v1` tag.

### Step 5: Push the image to Docker Hub

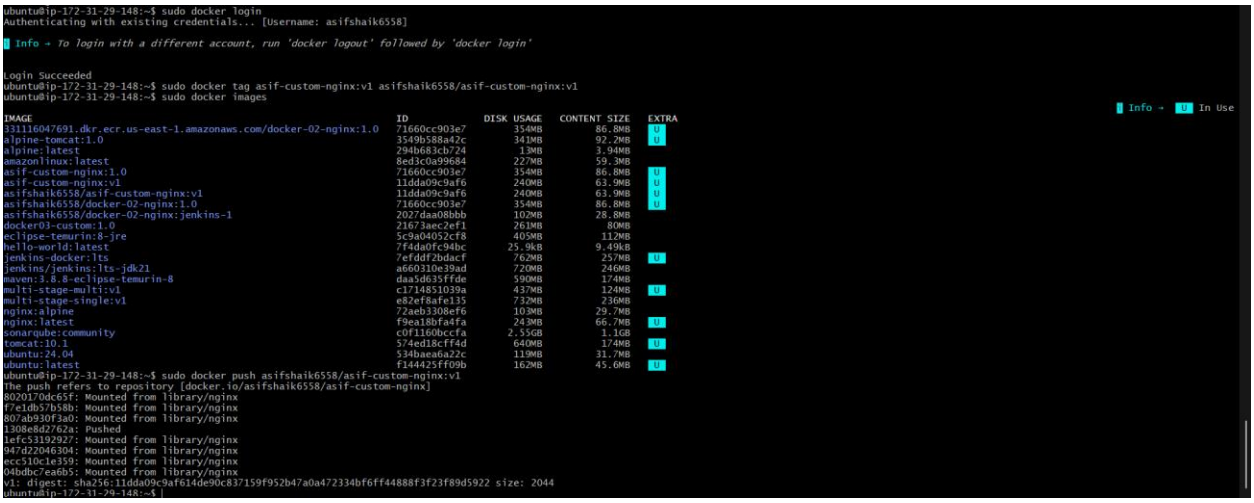
Command:

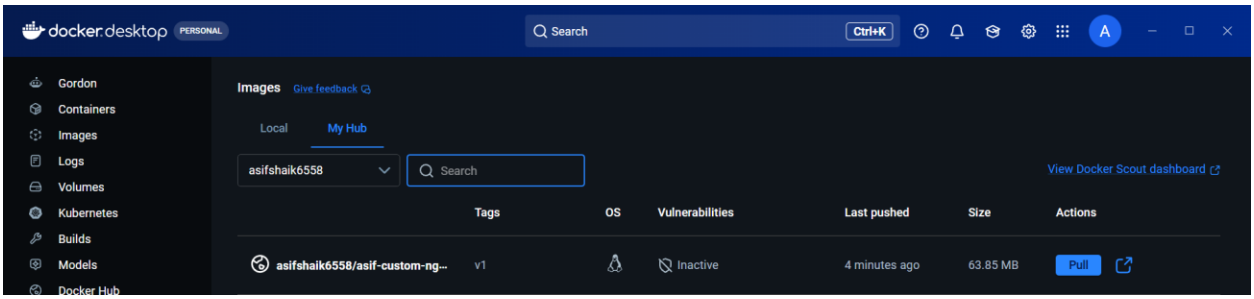
```
sudo docker push YOUR_DOCKERHUB_USERNAME/asif-custom-nginx:v1
```

Uploaded the customized image to the Docker Hub repository.

### Step 6: Verify the image in Docker Hub

Opened Docker Hub and checked the repository and its `v1` tag to confirm the upload.






### 4. Observations

- The customized Docker image was available locally on the EC2 instance.
- The image was tagged with the Docker Hub username and repository name.
- Docker Hub authentication was required before pushing the image.
- The image layers were uploaded to the remote registry when the push succeeded.
- The uploaded image could be shared and pulled by other systems, subject to repository visibility and access permissions.

### 5. Conclusion

Successfully prepared and pushed the customized Nginx Docker image to Docker Hub. This task demonstrated how to authenticate with a container registry, tag a local image, and publish it for remote access.

Docker Hub can now serve as the image source for other environments and automated deployment pipelines.

## Task 3: Create an Amazon ECR Repository for a Custom Docker Image

### 1. Objective

To create an Amazon Elastic Container Registry (ECR) repository in the AWS `us-east-1` region to store a customized Docker image and prepare it for uploading from an Ubuntu EC2 instance.

### 2. Why do we need to perform this task?

Amazon ECR is a managed container image registry used to store, manage, and distribute Docker images.

- Centralized storage: Store Docker images in an AWS repository.

- Security: Control repository access through IAM permissions.
- AWS integration: Make images available to AWS container deployment services.
- CI/CD automation: Allow Jenkins to push images to ECR as part of a deployment pipeline.
- Version management: Store different versions of an image using tags such as `v1` and `v2`.

## 3. Steps Used

### Step 1: Check whether the ECR repository exists

Command:

```
aws ecr describe-repositories \
  --repository-names asif-custom-nginx \
  --region us-east-1
```

Observation: The command returned `RepositoryNotFoundException` because the repository did not exist.

### Step 2: Attempt to create the ECR repository

Command:

```
aws ecr create-repository \
  --repository-name asif-custom-nginx \
  --region us-east-1
```

Observation: The command returned `AccessDeniedException` because the EC2 IAM role did not have permission to perform `ecr:CreateRepository`.

### Step 3: Update the IAM role permissions

Opened the AWS IAM Console and selected the role `EC2-ECR-Push-Role`.

Created an inline IAM policy granting the required ECR permissions, including repository creation, image upload, and registry authentication.

Policy name:

`ECR-Custom-Image-Push-Policy`

### Step 4: Create the ECR repository again

After updating the IAM permissions, executed:

```
aws ecr create-repository \
  --repository-name asif-custom-nginx \
  --region us-east-1
```

Observation: The repository was created successfully.

Repository details:

| Property | Value |
|---|---|
| Repository name | `asif-custom-nginx` |
| AWS region | `us-east-1` |
| AWS account ID | `331116047691` |
| Encryption | AES-256 |
| Image tag mutability | Mutable |
| Scan on push | Disabled |

Repository URI:

`331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx`

### Step 5: Prepare for Docker authentication and image upload

The next commands are ready to execute but have not yet been confirmed as successful.

Authenticate Docker:

```
aws ecr get-login-password --region us-east-1 | \
sudo docker login \
  --username AWS \
  --password-stdin 331116047691.dkr.ecr.us-east-1.amazonaws.com
```

Tag the existing local image:

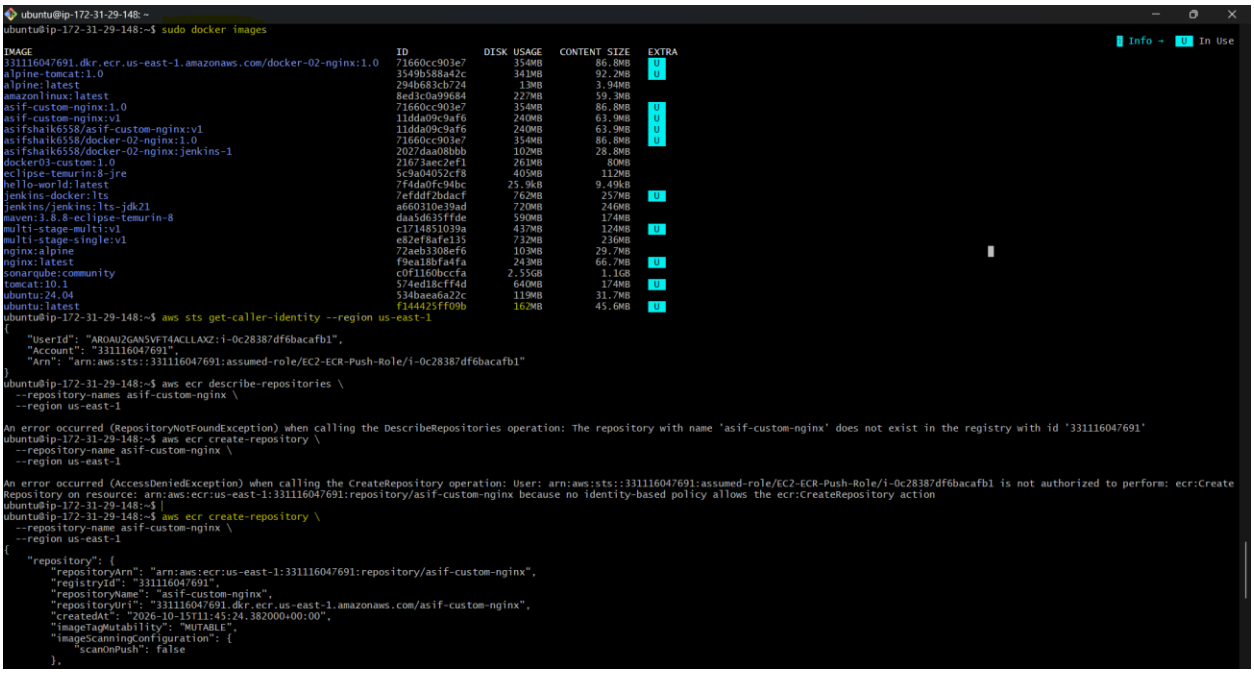
```
sudo docker tag asif-custom-nginx:v1 \
331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx:v1
```

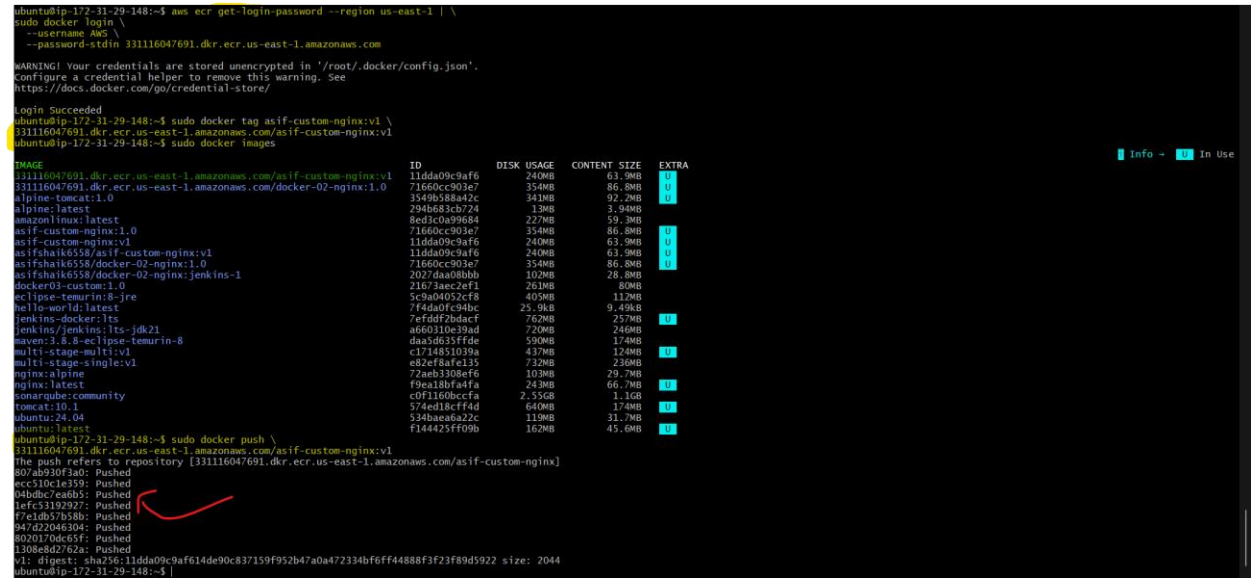
Push the image:

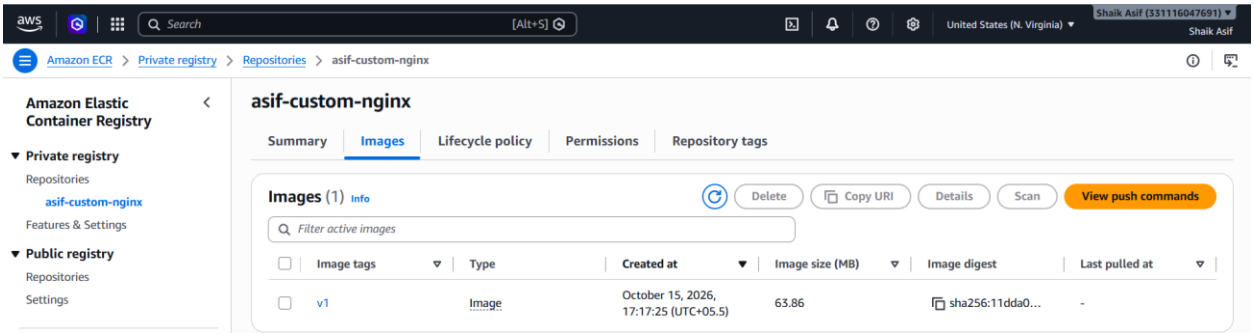
```
sudo docker push \
331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx:v1
```

Verify the uploaded image:

```
aws ecr describe-images \
  --repository-name asif-custom-nginx \
  --region us-east-1
```


ubuntu@ip-172-31-29-148: ~
ubuntu@ip-172-31-29-148:~$ sudo docker images
Info → U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0 71660cc903e7 354MB 86.8MB U
alpine-tomcat:1.0 3549b588a42c 341MB 92.2MB U
alpine:latest 294b683cb724 13MB 3.94MB
amazonlinux:latest 8ed3c0a99684 227MB 59.3MB
asif-custom-nginx:1.0 71660cc903e7 354MB 86.8MB U
asif-custom-nginx:v1 11dda09c9af6 240MB 63.9MB U
asifshaik6558/asif-custom-nginx:v1 11dda09c9af6 240MB 63.9MB U
asifshaik6558/docker-02-nginx:1.0 71660cc903e7 354MB 86.8MB U
asifshaik6558/docker-02-nginx:jenkins-1 2027daa08bbb 102MB 28.8MB
docker03-custom:1.0 21673aec2ef1 261MB 80MB
eclipse-temurin:8-jre 5c9a04052cf8 405MB 112MB
hello-world:latest 7f4da0fc94bc 25.9kB 9.49kB
jenkins-docker:lts 7efddf2bdacf 762MB 257MB U
jenkins/jenkins:lts-jdk21 a660310e39ad 720MB 246MB
maven:3.8.8-eclipse-temurin-8 daa5d635ffde 590MB 174MB
multi-stage-multi:v1 c1714851039a 437MB 124MB U
multi-stage-single:v1 e82ef8afe135 732MB 236MB
nginx:alpine 72aeb3308ef6 103MB 29.7MB
nginx:latest f9ea18bfa4fa 243MB 66.7MB U
sonarqube:community c0f1160bccfa 2.55GB 1.1GB
tomcat:10.1 574ed18cff4d 640MB 174MB U
ubuntu:24.04 534baea6a22c 119MB 31.7MB
ubuntu:latest f144425ff09b 162MB 45.6MB U
ubuntu@ip-172-31-29-148:~$ aws sts get-caller-identity --region us-east-1
{
"UserId": "AROAU2GAN5VFT4ACLLAXZ:i-0c28387df6bacafb1",
"Account": "331116047691",
"Arn": "arn:aws:sts::331116047691:assumed-role/EC2-ECR-Push-Role/i-0c28387df6bacafb1"
}
ubuntu@ip-172-31-29-148:~$ aws ecr describe-repositories \
--repository-names asif-custom-nginx \
--region us-east-1
An error occurred (RepositoryNotFoundException) when calling the DescribeRepositories operation: The repository with name 'asif-custom-nginx' does not exist in the registry with id '331116047691'
ubuntu@ip-172-31-29-148:~$ aws ecr create-repository \
--repository-name asif-custom-nginx \
--region us-east-1
An error occurred (AccessDeniedException) when calling the CreateRepository operation: User: arn:aws:sts::331116047691:assumed-role/EC2-ECR-Push-Role/i-0c28387df6bacafb1 is not authorized to perform: ecr:CreateRepository on resource: arn:aws:ecr:us-east-1:331116047691:repository/asif-custom-nginx because no identity-based policy allows the ecr:CreateRepository action
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ aws ecr create-repository \
--repository-name asif-custom-nginx \
--region us-east-1
{
"repository": {
"repositoryArn": "arn:aws:ecr:us-east-1:331116047691:repository/asif-custom-nginx",
"registryId": "331116047691",
"repositoryName": "asif-custom-nginx",
"repositoryUri": "331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx",
"createdAt": "2026-10-15T11:45:24.382000+00:00",
"imageTagMutability": "MUTABLE",
"imageScanningConfiguration": {
"scanOnPush": false
},


ubuntu@ip-172-31-29-148:~$ aws ecr get-login-password --region us-east-1 | \
sudo docker login \
--username AWS \
--password-stdin 331116047691.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
ubuntu@ip-172-31-29-148:~$ sudo docker tag asif-custom-nginx:v1 \
331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx:v1
ubuntu@ip-172-31-29-148:~$ sudo docker images
Info → U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx:v1 11dda09c9af6 240MB 63.9MB U
331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0 71660cc903e7 354MB 86.8MB U
alpine-tomcat:1.0 3549b588a42c 341MB 92.2MB U
alpine:latest 294b683cb724 13MB 3.94MB
amazonlinux:latest 8ed3c0a99684 227MB 59.3MB
asif-custom-nginx:1.0 71660cc903e7 354MB 86.8MB U
asif-custom-nginx:v1 11dda09c9af6 240MB 63.9MB U
asifshaik6558/asif-custom-nginx:v1 11dda09c9af6 240MB 63.9MB U
asifshaik6558/docker-02-nginx:1.0 71660cc903e7 354MB 86.8MB U
asifshaik6558/docker-02-nginx:jenkins-1 2027daa08bbb 102MB 28.8MB
docker03-custom:1.0 21673aec2ef1 261MB 80MB
eclipse-temurin:8-jre 5c9a04052cf8 405MB 112MB
hello-world:latest 7f4da0fc94bc 25.9kB 9.49kB
jenkins-docker:lts 7efddf2bdacf 762MB 257MB U
jenkins/jenkins:lts-jdk21 a660310e39ad 720MB 246MB
maven:3.8.8-eclipse-temurin-8 daa5d635ffde 590MB 174MB
multi-stage-multi:v1 c1714851039a 437MB 124MB U
multi-stage-single:v1 e82ef8afe135 732MB 236MB
nginx:alpine 72aeb3308ef6 103MB 29.7MB
nginx:latest f9ea18bfa4fa 243MB 66.7MB U
sonarqube:community c0f1160bccfa 2.55GB 1.1GB
tomcat:10.1 574ed18cff4d 640MB 174MB U
ubuntu:24.04 534baea6a22c 119MB 31.7MB
ubuntu:latest f144425ff09b 162MB 45.6MB U
ubuntu@ip-172-31-29-148:~$ sudo docker push \
331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx:v1
The push refers to repository [331116047691.dkr.ecr.us-east-1.amazonaws.com/asif-custom-nginx]
807ab930f3a0: Pushed
ecc510c1e359: Pushed
04bdbc7ea6b5: Pushed
1efc53192927: Pushed
f7e1db57b58b: Pushed
947d22046304: Pushed
8020170dc65f: Pushed
1308e8d2762a: Pushed
v1: digest: sha256:11dda09c9af614de90c837159f952b47a0a472334bf6ff44888f3f23f89d5922 size: 2044
ubuntu@ip-172-31-29-148:~$


aws
Search
[Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
Amazon ECR > Private registry > Repositories > asif-custom-nginx
Amazon Elastic Container Registry
Private registry
Repositories
asif-custom-nginx
Features & Settings
Public registry
Repositories
Settings
asif-custom-nginx
Summary
Images
Lifecycle policy
Permissions
Repository tags
Images (1) Info
Delete
Copy URI
Details
Scan
View push commands
Filter active images
Image tags
Type
Created at
Image size (MB)
Image digest
Last pulled at
v1
Image
October 15, 2026, 17:17:25 (UTC+05.5)
63.86
sha256:11dda0...
-

### 4. Observations

- The ECR repository was initially absent.
- Repository creation failed because the EC2 IAM role lacked the required permission.
- After updating the IAM role permissions, repository creation succeeded.
- The repository was created in `us-east-1` with AES-256 encryption.
- The repository is ready to receive the customized Docker image.
- Image upload and verification remain pending until the authentication and push commands succeed.

### 5. Conclusion

Successfully created the Amazon ECR repository `asif-custom-nginx` after identifying and resolving an IAM permission issue. This task demonstrated how IAM permissions control access to AWS container registries and how ECR provides a centralized location for storing Docker images.

Once the existing `asif-custom-nginx:v1` image is pushed successfully, it will be available in ECR for use in AWS-based deployment workflows and CI/CD pipelines.

## Task 4: Terraform EC2 Provisioning and Jenkins Installation

### Objective

To use Terraform to provision a new AWS EC2 instance, install Jenkins automatically through `user_data`, access Jenkins through a browser, and verify pipeline execution.

### Step 1: Navigate to the Task 4 directory

Connected to the existing Ubuntu EC2 instance and navigated to the assignment directory.

```
cd ~/docker-challenge/task-04
```

### Step 2: Create the Terraform configuration

Opened the Terraform file:

```
nano main.tf
```

Initially, the file contained the AWS provider configuration:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

### Step 3: Configure the EC2 instance and Jenkins installation

Added the following complete Terraform configuration to `main.tf`.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "terraform_jenkins" {
  ami           = "ami-0c7217cdde317cfec"
  instance_type = "t3.micro"
  key_name      = "jenkins"

  vpc_security_group_ids = ["sg-0ae1da12bd9cb8dcf"]

  user_data = <<-EOF
    #!/bin/bash
    set -e

    apt-get update -y
    apt-get install -y fontconfig openjdk-21-jre wget gnupg

    mkdir -p /etc/apt/keyrings

    wget -qO /etc/apt/keyrings/jenkins-keyring.asc \
      https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

    echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" \
      > /etc/apt/sources.list.d/jenkins.list

    apt-get update -y
    apt-get install -y jenkins

    systemctl enable --now jenkins
  EOF

  tags = {
    Name        = "terraform-jenkins"
    Environment = "Docker-Challenge"
  }
}

output "instance_id" {
```

```
  description = "ID of the Terraform-created EC2 instance"
  value       = aws_instance.terraform_jenkins.id
}

output "public_ip" {
  description = "Public IP address of the new EC2 instance"
  value       = aws_instance.terraform_jenkins.public_ip
}
```

Saved and exited Nano using:

- `Ctrl + O` — save the file
- `Enter` — confirm the filename
- `Ctrl + X` — exit

### What the configuration did

- `provider "aws"` selected the AWS region `us-east-1`.
- `aws_instance` defined the EC2 instance to create.
- `ami` specified the Ubuntu machine image.
- `instance_type` selected `t3.micro`.
- `key_name` specified the existing `jenkins` key pair.
- `vpc_security_group_ids` attached the specified security group.
- `user_data` supplied a startup script to install Java 21 and Jenkins.
- `tags` assigned the instance name and environment.
- `output` displayed the new instance ID and public IP after provisioning.

### Step 4: Format, validate, and plan

Executed these commands:

```
terraform fmt
terraform validate
terraform plan
```

Observation: Terraform reported:

```
Success! The configuration is valid.
Plan: 1 to add, 0 to change, 0 to destroy.
```

This confirmed that Terraform planned to create one instance without changing or destroying existing resources.

### Step 5: Attempt to provision the EC2 instance

Executed:

```
terraform apply
```

The first attempt failed with:

```
UnauthorizedOperation
not authorized to perform: ec2:RunInstances
```

Reason: The IAM role attached to the existing EC2 instance, `EC2-ECR-Push-Role`, did not have permission to launch another EC2 instance.

### Step 6: Fix the IAM permission error

Opened the AWS IAM Console:

1. Selected Roles.
2. Opened `EC2-ECR-Push-Role`.
3. Selected Add permissions → Create inline policy.
4. Opened the JSON tab.
5. Added the following policy.
6. Named it `Terraform-EC2-Launch-Policy` and created it.

Policy used:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:RunInstances",
        "ec2:DescribeInstances",
        "ec2:DescribeImages",
        "ec2:DescribeInstanceTypes",
        "ec2:DescribeKeyPairs",
        "ec2:DescribeSecurityGroups",
```

```
        "ec2:DescribeSubnets",
        "ec2:DescribeVpcs",
        "ec2:CreateTags"
      ],
      "Resource": "*"
    }
  ]
}
```

Observation: The IAM role was updated to include the EC2 launch permissions required for the next provisioning attempt.

### Step 7: Provision the EC2 instance successfully

Ran the command again:

```
terraform apply
```

At the confirmation prompt, entered:

```
yes
```

Terraform returned:

```
aws_instance.terraform_jenkins: Creation complete
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

The outputs were:

```
instance_id = "i-0e8b0faf7c9f0acbb"
public_ip = "54.227.93.199"
```

Observation: AWS successfully created the new EC2 instance.

### Step 8: Connect to the new EC2 instance

Opened Git Bash on the Windows laptop and executed:

```
ssh -i jenkins.pem ubuntu@54.227.93.199
```

The SSH connection succeeded and displayed the Ubuntu 22.04.3 LTS welcome message.

The new server's prompt was:

```
ubuntu@ip-172-31-27-153:~$
```

### Step 9: Verify Jenkins installation

On the new EC2 instance, executed:

```
sudo systemctl status jenkins --no-pager
```

The output confirmed:

```
jenkins.service - Jenkins Continuous Integration Server
Active: active (running)
```

Observation: Jenkins was installed and running as a Linux system service.

### Step 10: Access Jenkins in the browser

Opened the following URL:

http://54.227.93.199:8080

The Jenkins initial setup page appeared. Completed the setup and selected Start using Jenkins.

Observation: The Jenkins dashboard opened successfully and was ready to use.

### Step 11: Create a Declarative Pipeline

Created a new Jenkins job:

1. Clicked New Item.
2. Entered `Terraform-Pipeline-Test`.
3. Selected Pipeline.
4. Clicked OK.
5. Scrolled to the Pipeline section.
6. Selected Pipeline script under Definition.
7. Entered the following script and saved the job.

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
```

```
                echo 'Build stage started'
                echo 'Application build completed'
            }
        }

        stage('Test') {
            steps {
                echo 'Testing started'
                echo 'All tests passed'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deployment completed successfully'
            }
        }
    }
}
```

Clicked Build Now and opened Console Output.

Observation: The Build, Test, and Deploy stages executed successfully. The console ended with:

```
Finished: SUCCESS
```

### Step 12: Create and execute a Parallel Pipeline

Opened `Terraform-Pipeline-Test`, selected Configure, and replaced the pipeline script with:

```
pipeline {
    agent any

    stages {
        stage('Parallel Tasks') {
            parallel {
                stage('Build') {
                    steps {
                        echo 'Building application...'
                    }
```

```
                }

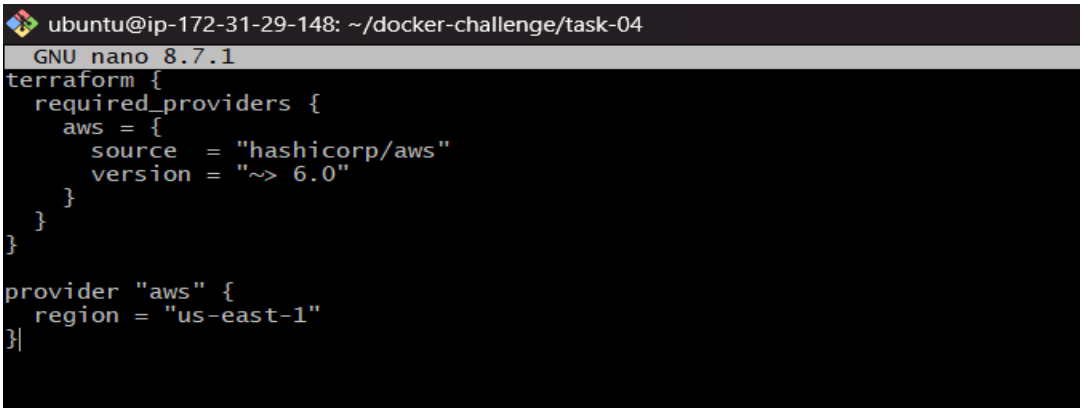
                stage('Test') {
                    steps {
                        echo 'Testing application...'
                    }
                }
            }
        }

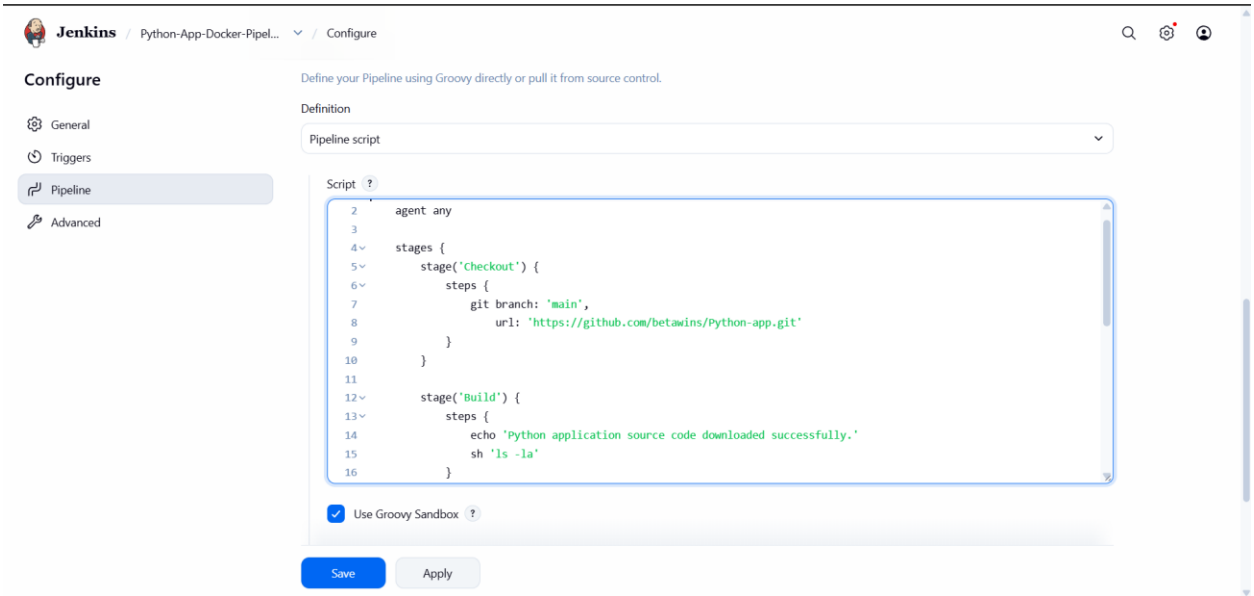
        stage('Deploy') {
            steps {
                echo 'Deployment completed'
            }
        }
    }
}
```

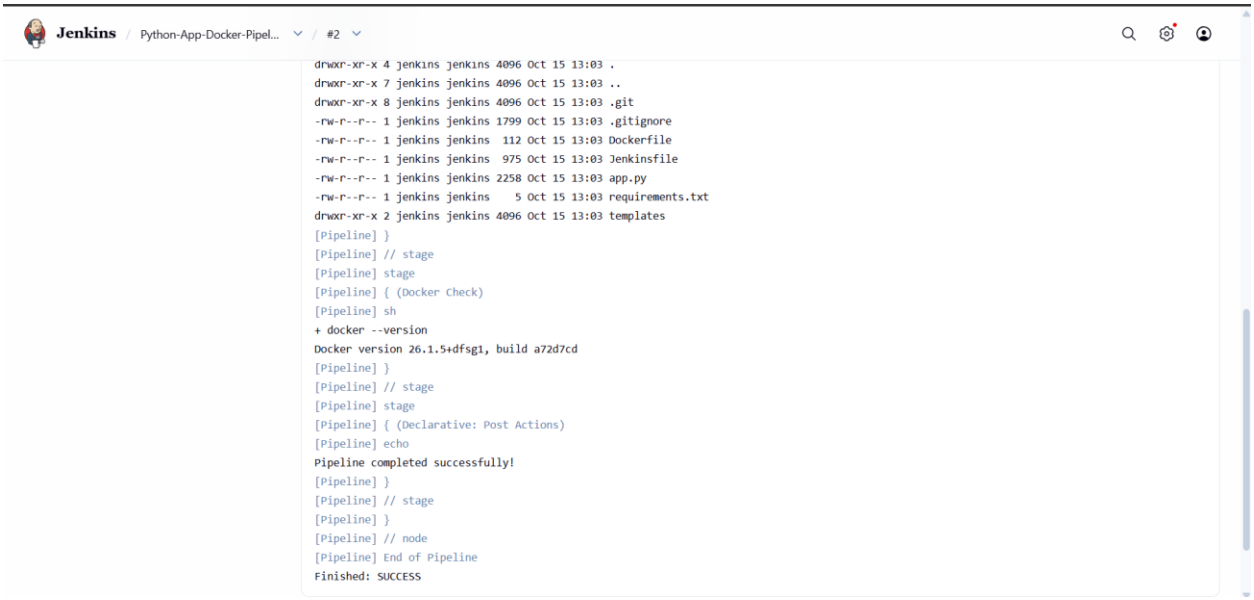
Saved the job, clicked Build Now, and checked Console Output.

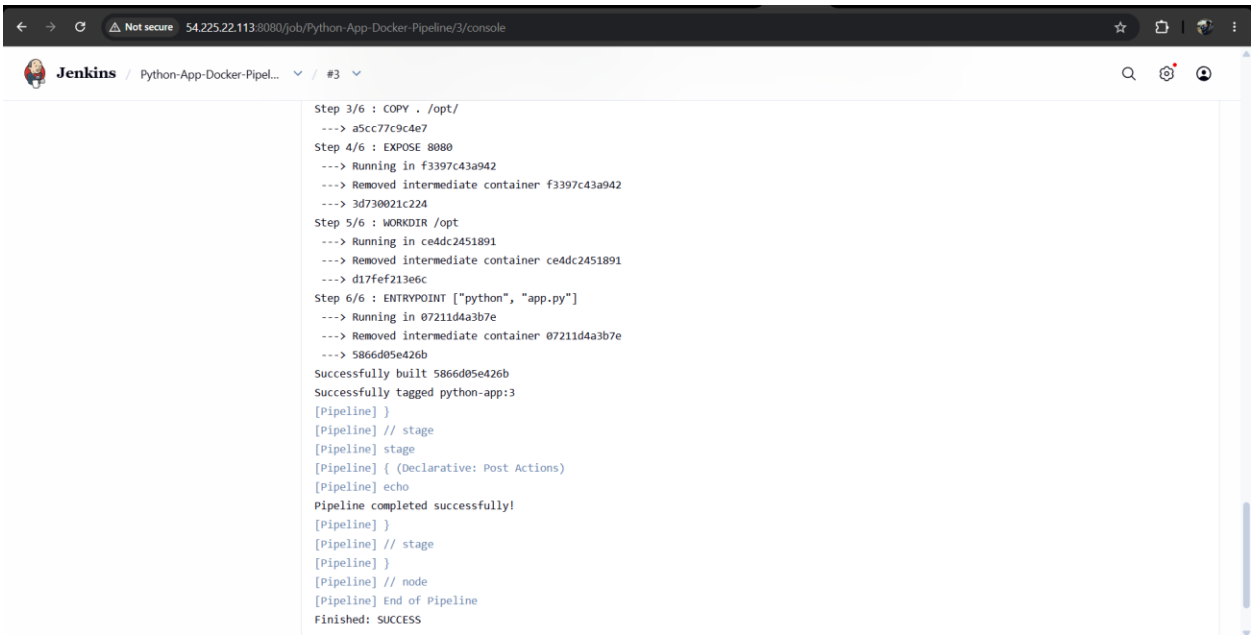
Observation: The Build and Test branches executed in parallel. The Deploy stage executed after both branches completed. The console ended with:

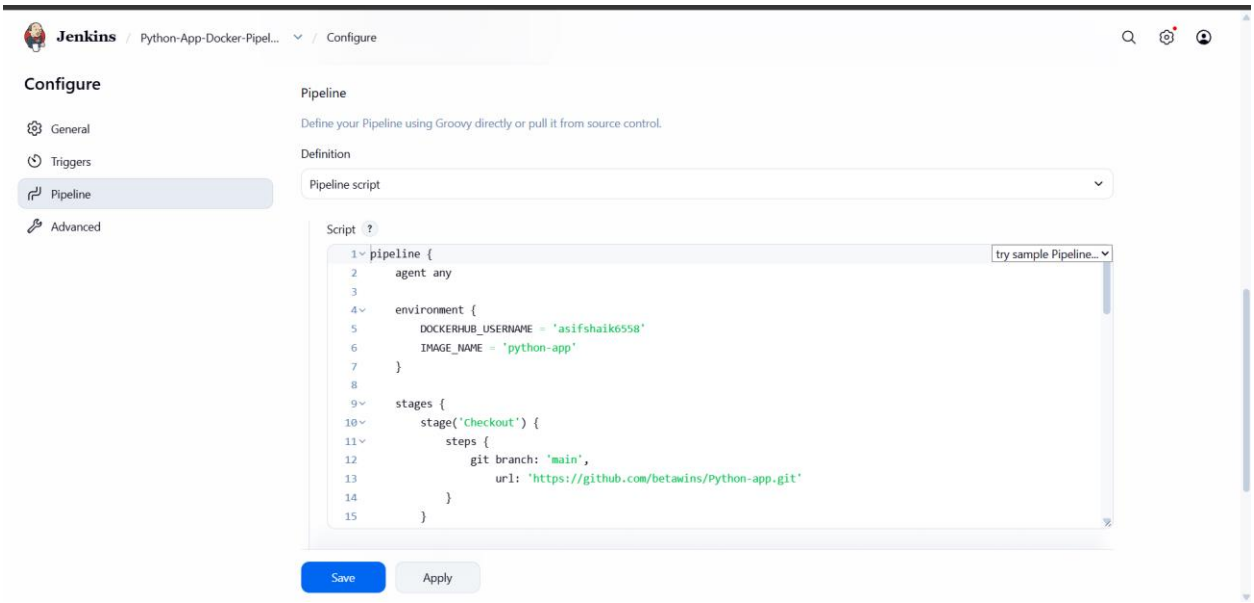
```
Finished: SUCCESS
```

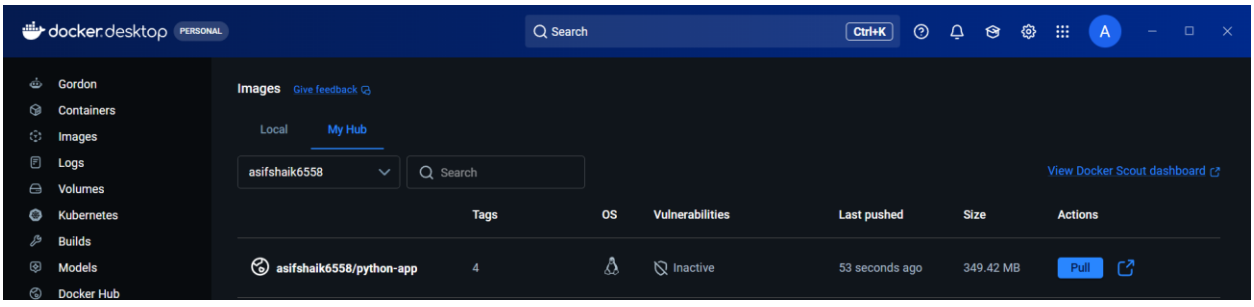


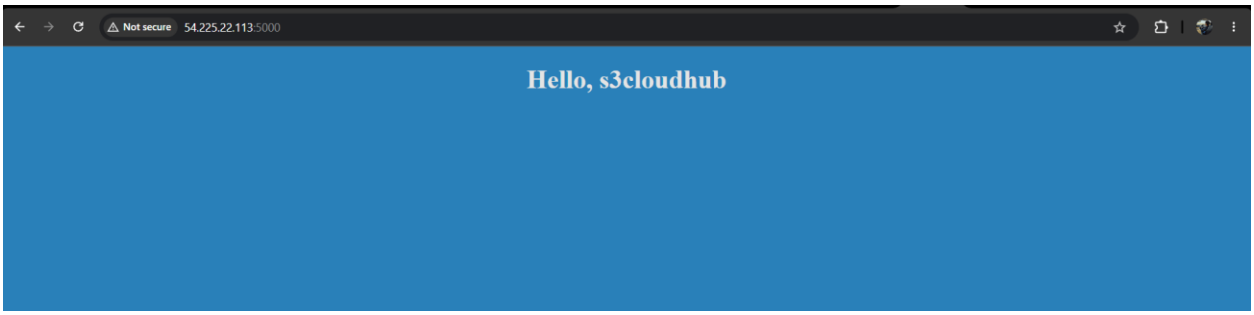

Jenkins / Python-App-Docker-Pipel... / Configure
Configure
General
Triggers
Pipeline
Advanced
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
2 agent any
3
4 stages {
5 stage('Checkout') {
6 steps {
7 git branch: 'main',
8 url: 'https://github.com/betawins/Python-app.git'
9 }
10 }
11
12 stage('Build') {
13 steps {
14 echo 'Python application source code downloaded successfully.'
15 sh 'ls -la'
16 }
Use Groovy Sandbox
Save
Apply


Jenkins / Python-App-Docker-Pipel... / #2
drwxr-xr-x 4 jenkins jenkins 4096 Oct 15 13:03 .
drwxr-xr-x 7 jenkins jenkins 4096 Oct 15 13:03 ..
drwxr-xr-x 8 jenkins jenkins 4096 Oct 15 13:03 .git
-rw-r--r-- 1 jenkins jenkins 1799 Oct 15 13:03 .gitignore
-rw-r--r-- 1 jenkins jenkins 112 Oct 15 13:03 Dockerfile
-rw-r--r-- 1 jenkins jenkins 975 Oct 15 13:03 Jenkinsfile
-rw-r--r-- 1 jenkins jenkins 2258 Oct 15 13:03 app.py
-rw-r--r-- 1 jenkins jenkins 5 Oct 15 13:03 requirements.txt
drwxr-xr-x 2 jenkins jenkins 4096 Oct 15 13:03 templates
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Docker Check)
[Pipeline] sh
+ docker --version
Docker version 26.1.5+dfsg1, build a72d7cd
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure 54.225.22.113:8080/job/Python-App-Docker-Pipeline/3/console
Jenkins / Python-App-Docker-Pipel... / #3
Step 3/6 : COPY . /opt/
---> a5cc77c9c4e7
Step 4/6 : EXPOSE 8080
---> Running in f3397c43a942
---> Removed intermediate container f3397c43a942
---> 3d730021c224
Step 5/6 : WORKDIR /opt
---> Running in ce4dc2451891
---> Removed intermediate container ce4dc2451891
---> d17fef213e6c
Step 6/6 : ENTRYPOINT ["python", "app.py"]
---> Running in 07211d4a3b7e
---> Removed intermediate container 07211d4a3b7e
---> 5866d05e426b
Successfully built 5866d05e426b
Successfully tagged python-app:3
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Jenkins / Python-App-Docker-Pipel... / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
try sample Pipeline...
pipeline {
agent any
environment {
DOCKERHUB_USERNAME = 'asifshaik6558'
IMAGE_NAME = 'python-app'
}
stages {
stage('Checkout') {
steps {
git branch: 'main',
url: 'https://github.com/betawins/Python-app.git'
}
}
Save
Apply


docker desktop PERSONAL
Search Ctrl+K
Gordon
Containers
Images
Logs
Volumes
Kubernetes
Builds
Models
Docker Hub
Images Give feedback
Local
My Hub
asifshaik6558
Search
View Docker Scout dashboard
Tags
OS
Vulnerabilities
Last pushed
Size
Actions
asifshaik6558/python-app
4
Inactive
53 seconds ago
349.42 MB
Pull




ubuntu@ip-172-31-29-148: ~/docker-challenge/task-04

GNU nano 8.7.1 main.tf *

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

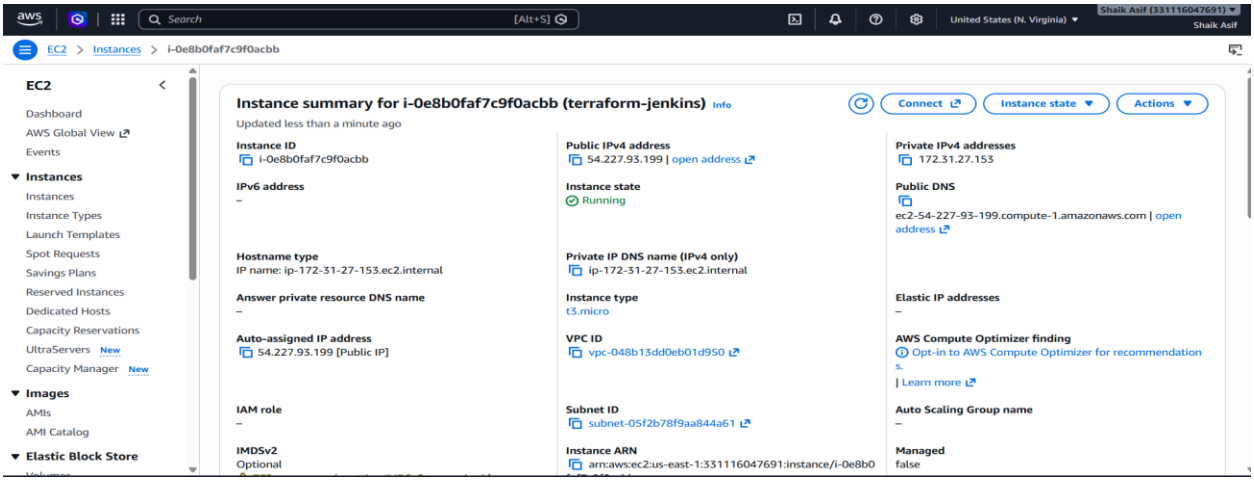
provider "aws" {
  region = "us-east-1"
}

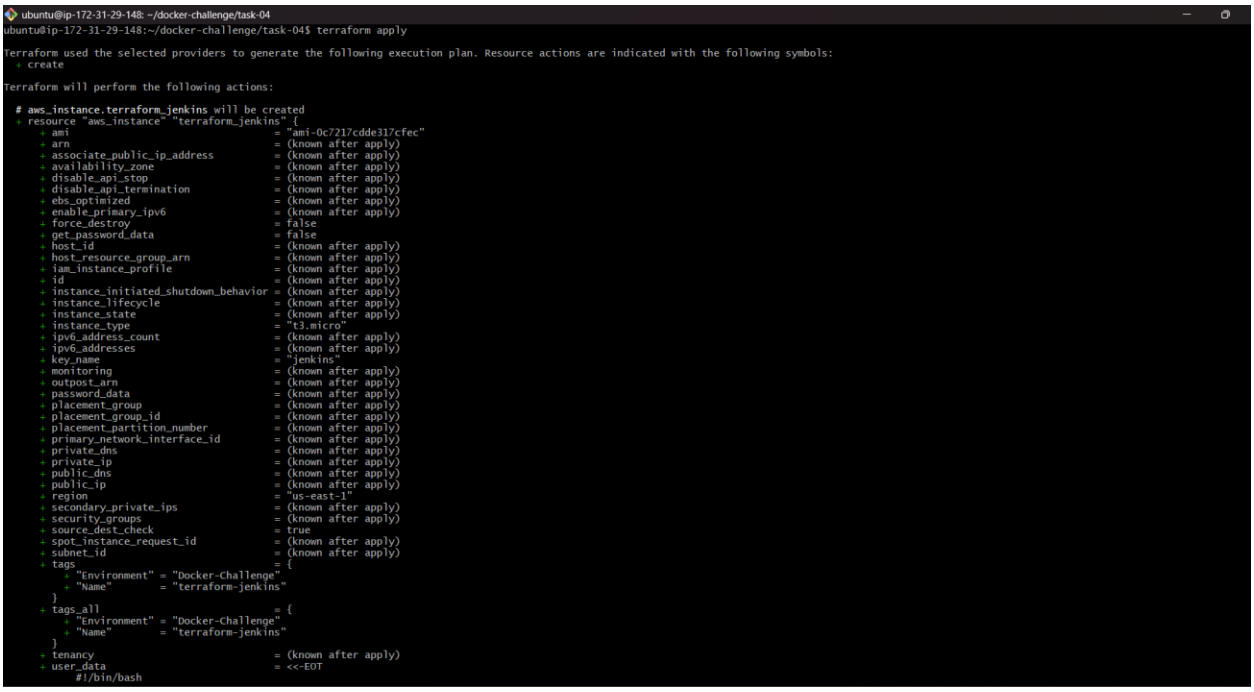
resource "aws_instance" "terraform_jenkins" {
  ami           = "ami-0c7217cdde317cfec"
  instance_type = "t3.micro"

  vpc_security_group_ids = ["sg-0ae1da12bd9cb8dcf"]

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y fontconfig openjdk-21-jre
    EOF

  tags = {
    Name        = "terraform-jenkins"
    Environment = "Docker-Challenge"
  }
}
```



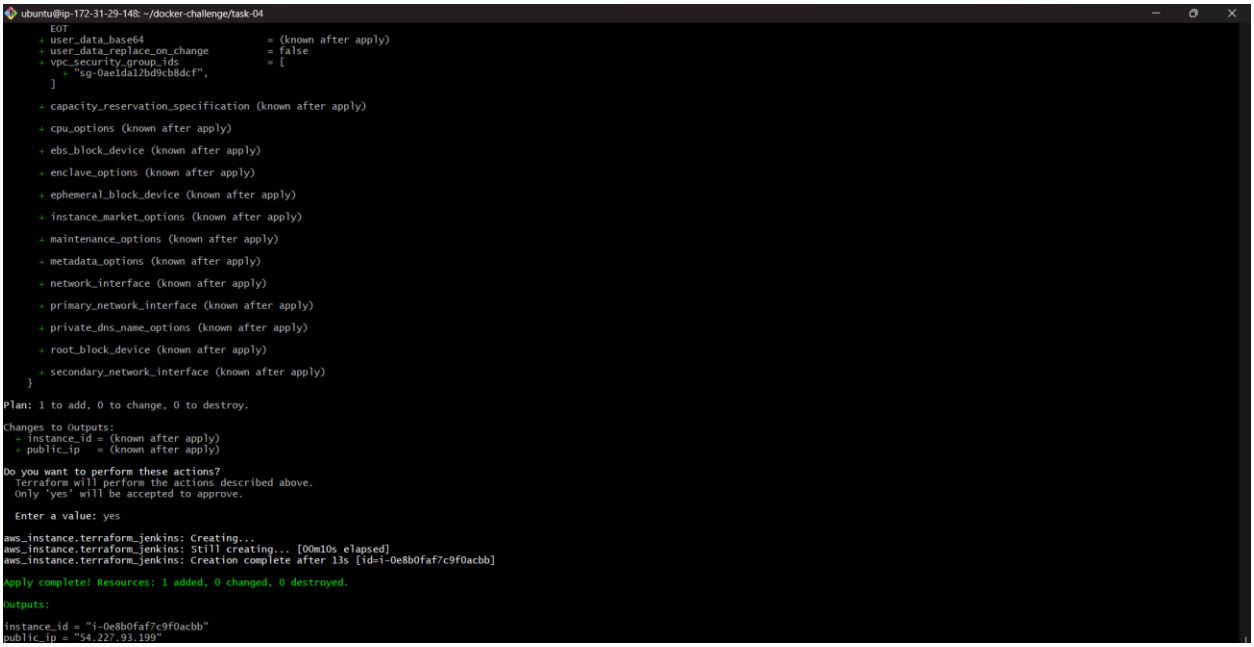


```
ubuntu@ip-172-31-29-148: ~/docker-challenge/task-04
ubuntu@ip-172-31-29-148:~/docker-challenge/task-04$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.terraform_jenkins will be created
  + resource "aws_instance" "terraform_jenkins" {
      + ami                                  = "ami-0c7217cdde317cfec"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "jenkins"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "us-east-1"
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = (known after apply)
      + tags                                 = {
          + "Environment" = "Docker-Challenge"
          + "Name"        = "terraform-jenkins"
        }
      + tags_all                             = {
          + "Environment" = "Docker-Challenge"
          + "Name"        = "terraform-jenkins"
        }
      + tenancy                              = (known after apply)
      + user_data                            = <<-EOT
            #!/bin/bash
```



```
ubuntu@ip-172-31-29-148: ~/docker-challenge/task-04
        EOT
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = [
          + "sg-0ae1da12bd9cb8dcf",
        ]

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id = (known after apply)
  + public_ip   = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

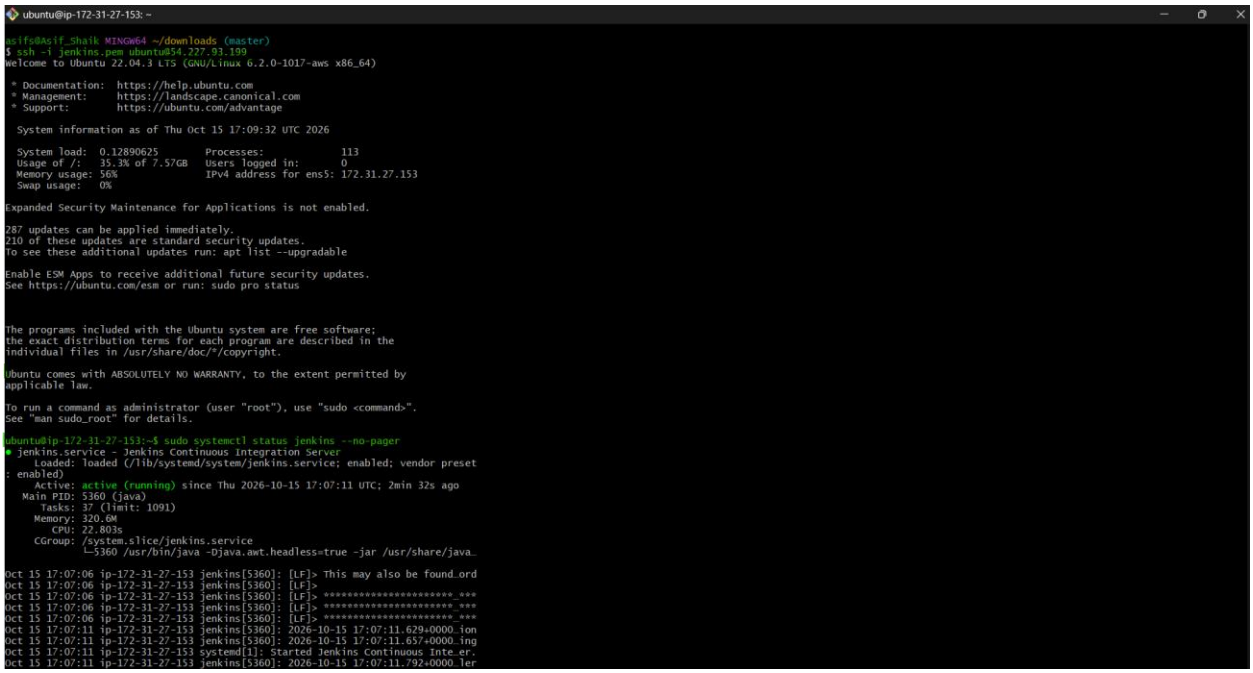
  Enter a value: yes

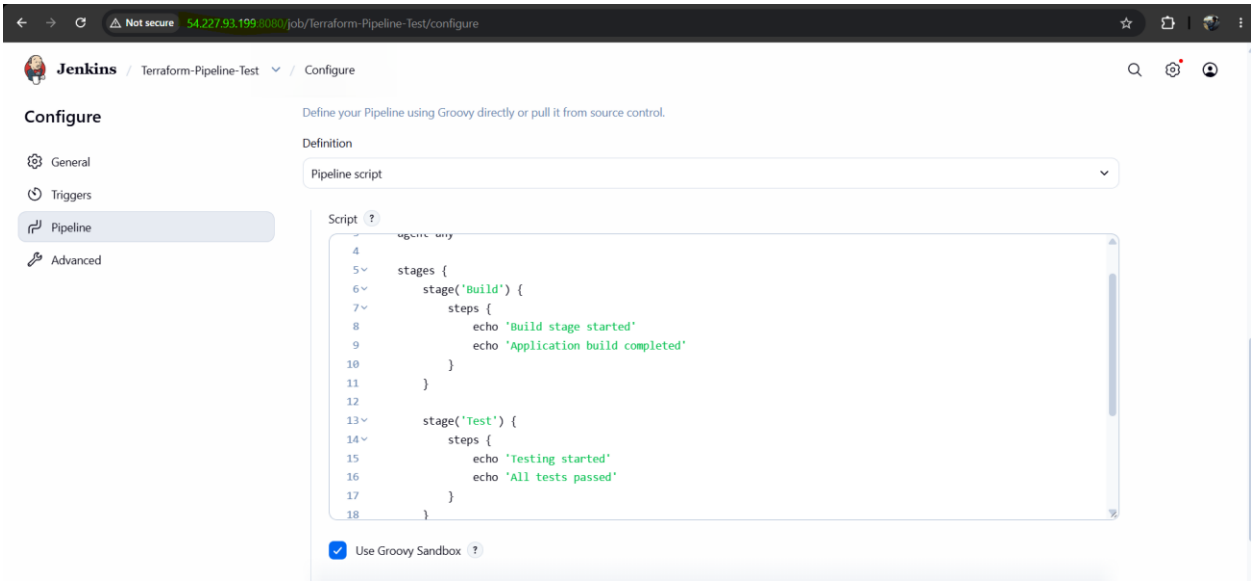
aws_instance.terraform_jenkins: Creating...
aws_instance.terraform_jenkins: Still creating... [00m10s elapsed]
aws_instance.terraform_jenkins: Creation complete after 13s [id=i-0e8b0faf7c9f0acbb]

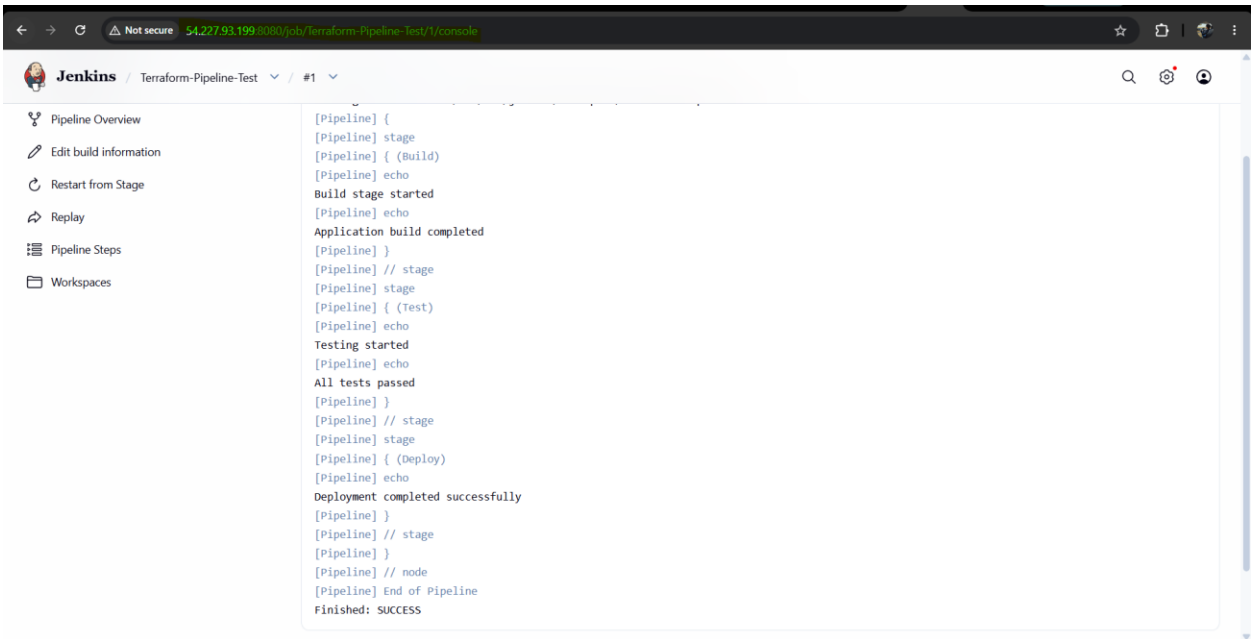
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

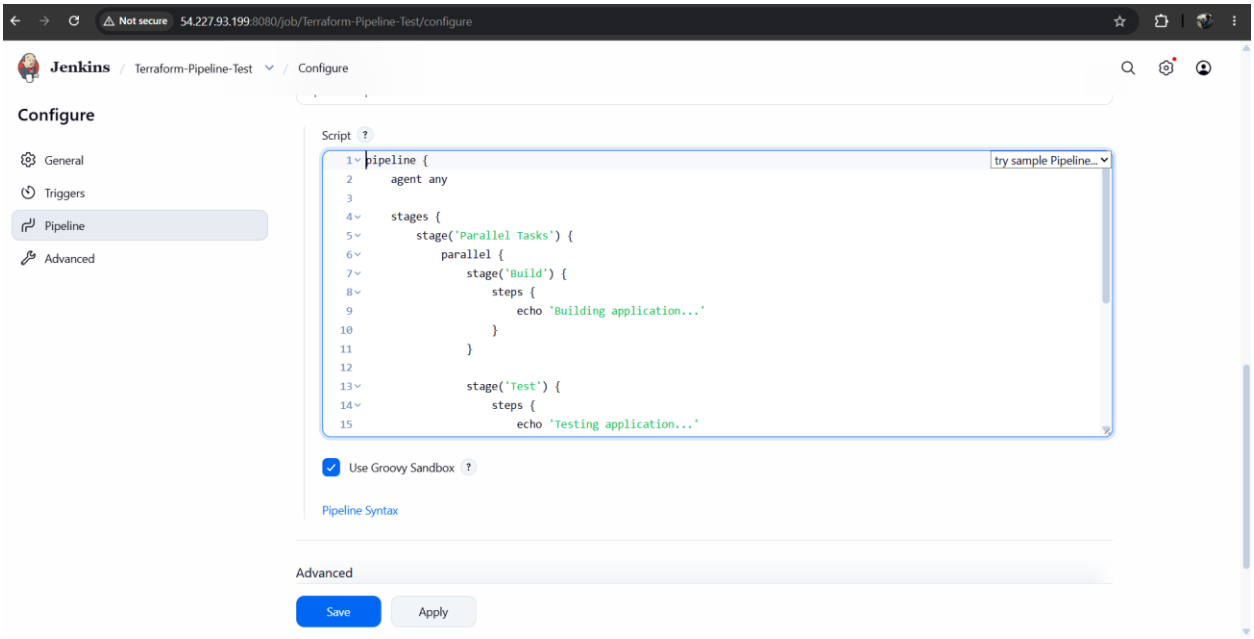
Outputs:

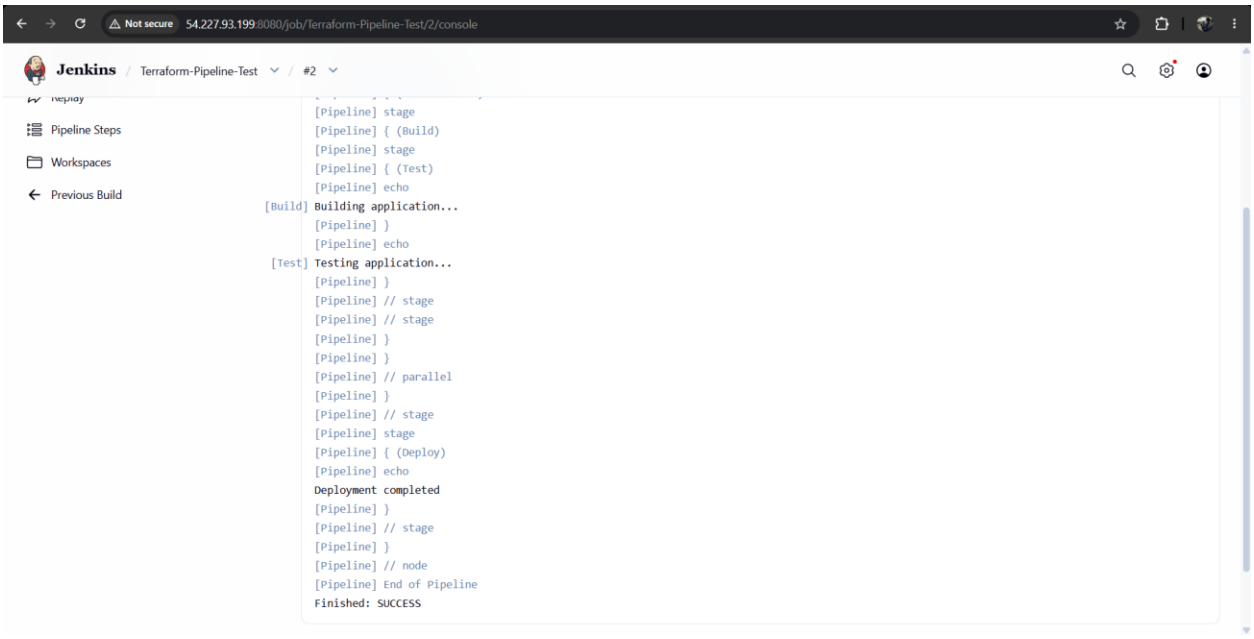
instance_id = "i-0e8b0faf7c9f0acbb"
public_ip = "54.227.93.199"
```


ubuntu@ip-172-31-27-153: ~
asifs@Asif_Shaik MINGW64 ~/downloads (master)
$ ssh -i jenkins.pem ubuntu@54.227.93.199
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.2.0-1017-aws x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage
System information as of Thu Oct 15 17:09:32 UTC 2026
System load: 0.12890625 Processes: 113
Usage of /: 35.3% of 7.57GB Users logged in: 0
Memory usage: 56% IPv4 address for ens5: 172.31.27.153
Swap usage: 0%
Expanded Security Maintenance for Applications is not enabled.
287 updates can be applied immediately.
210 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
ubuntu@ip-172-31-27-153:~$ sudo systemctl status jenkins --no-pager
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; vendor preset
: enabled)
Active: active (running) since Thu 2026-10-15 17:07:11 UTC; 2min 32s ago
Main PID: 5360 (java)
Tasks: 37 (limit: 1091)
Memory: 320.6M
CPU: 22.803s
CGroup: /system.slice/jenkins.service
└─5360 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java…
Oct 15 17:07:06 ip-172-31-27-153 jenkins[5360]: [LF]> This may also be found…ord
Oct 15 17:07:06 ip-172-31-27-153 jenkins[5360]: [LF]>
Oct 15 17:07:06 ip-172-31-27-153 jenkins[5360]: [LF]> **********…***
Oct 15 17:07:06 ip-172-31-27-153 jenkins[5360]: [LF]> **********…***
Oct 15 17:07:06 ip-172-31-27-153 jenkins[5360]: [LF]> **********…***
Oct 15 17:07:11 ip-172-31-27-153 jenkins[5360]: 2026-10-15 17:07:11.629+0000…ion
Oct 15 17:07:11 ip-172-31-27-153 jenkins[5360]: 2026-10-15 17:07:11.657+0000…ing
Oct 15 17:07:11 ip-172-31-27-153 systemd[1]: Started Jenkins Continuous Inte…er.
Oct 15 17:07:11 ip-172-31-27-153 jenkins[5360]: 2026-10-15 17:07:11.792+0000…ler


Not secure 54.227.93.199:8080/job/Terraform-Pipeline-Test/configure
Jenkins / Terraform-Pipeline-Test / Configure
Configure
General
Triggers
Pipeline
Advanced
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
stages {
stage('Build') {
steps {
echo 'Build stage started'
echo 'Application build completed'
}
}
stage('Test') {
steps {
echo 'Testing started'
echo 'All tests passed'
}
}
Use Groovy Sandbox


Not secure 54.227.93.199:8080/job/Terraform-Pipeline-Test/1/console
Jenkins / Terraform-Pipeline-Test / #1
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build stage started
[Pipeline] echo
Application build completed
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing started
[Pipeline] echo
All tests passed
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deployment completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure 54.227.93.199:8080/job/Terraform-Pipeline-Test/configure
Jenkins / Terraform-Pipeline-Test / Configure
Configure
General
Triggers
Pipeline
Advanced
Script
try sample Pipeline...
pipeline {
agent any
stages {
stage('Parallel Tasks') {
parallel {
stage('Build') {
steps {
echo 'Building application...'
}
}
stage('Test') {
steps {
echo 'Testing application...'
Use Groovy Sandbox
Pipeline Syntax
Advanced
Save
Apply




## Final observation

- Terraform formatted and validated the configuration successfully.
- The first provisioning attempt failed because the IAM role lacked `ec2:RunInstances`.
- After updating the IAM permissions, Terraform created the EC2 instance successfully.
- Jenkins was installed automatically through `user_data`.
- SSH access to the new instance was successful.
- Jenkins was verified as `active (running)`.
- The Jenkins initial setup was completed.
- Both the Declarative and Parallel Pipelines completed successfully.
- The existing Jenkins server and application were not modified by the Terraform configuration.

## Conclusion

Successfully provisioned a new EC2 instance using Terraform, resolved an IAM authorization error, automated Jenkins installation using `user_data`, and verified Jenkins operation by executing Declarative and Parallel Pipelines.

# 1. Create one Jenkins job to build and push the Docker image to Docker Hub.

a. **Source: https://github.com/betawins/Python-app.git**



**Source Codes: https://github.com/betawins/docker-tasks.git**

- **From the frontend source code, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.**
- **From the Java-based source code, write a Dockerfile, build, run, and push the image to the Docker registry.**
- **From the Node.js-based source code, write a Dockerfile, build with tag `v1`, run, and push it to the Docker registry.**
- **Write a docker-compose file to set up WordPress with a MySQL database.**

## Jenkins Pipeline — Build and Push Python Application

### Objective

To automate the process of pulling Python application source code from GitHub, building a Docker image, and pushing it to Docker Hub using a Jenkins Declarative Pipeline.

### Steps Used

1. Started the Jenkins container and accessed Jenkins through the browser.
2. Created/configured the Jenkins job `Python-App-Docker-Pipeline`.
3. Configured Git checkout using the repository: https://github.com/betawins/Python-app.git
4. Used the `main` branch to retrieve the source code.
5. Configured Docker Hub credentials in Jenkins using credentials ID `dockerhub-creds`.
6. Built the Docker image using the Jenkins build number as the tag.
7. Logged in to Docker Hub through Jenkins, pushed the image, and logged out.
8. Executed the pipeline and checked the console output.

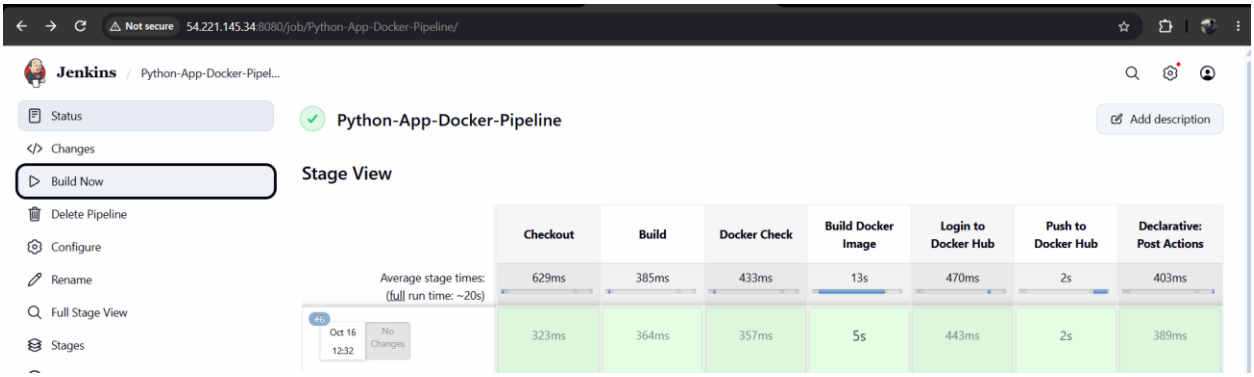
Key pipeline commands used:

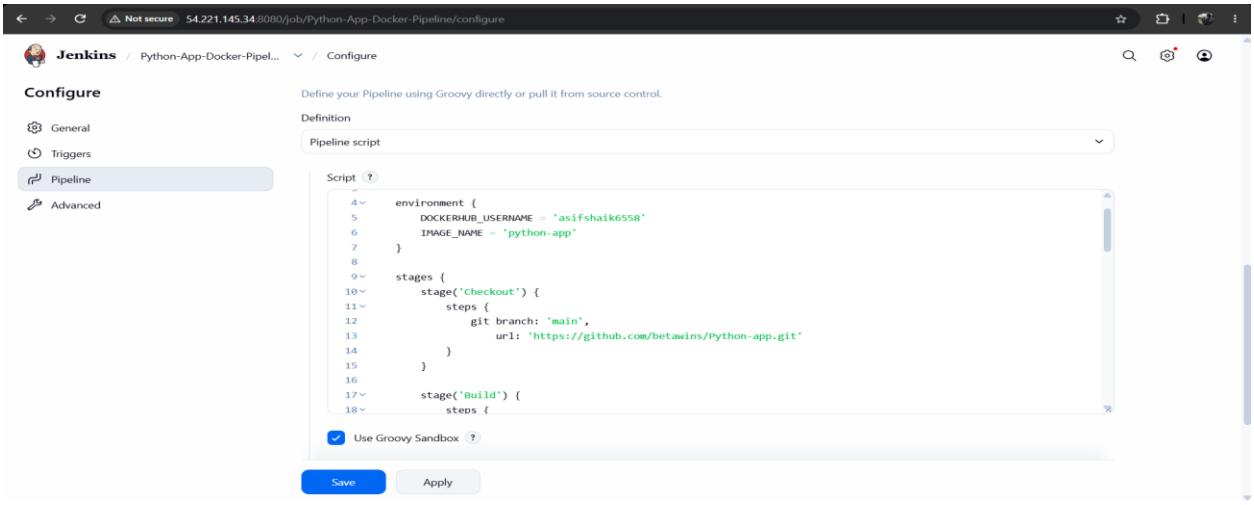
```
checkout scmGit(
    branches: [[name: '*/main']],
    userRemoteConfigs: [[
        url: 'https://github.com/betawins/Python-app.git'
```

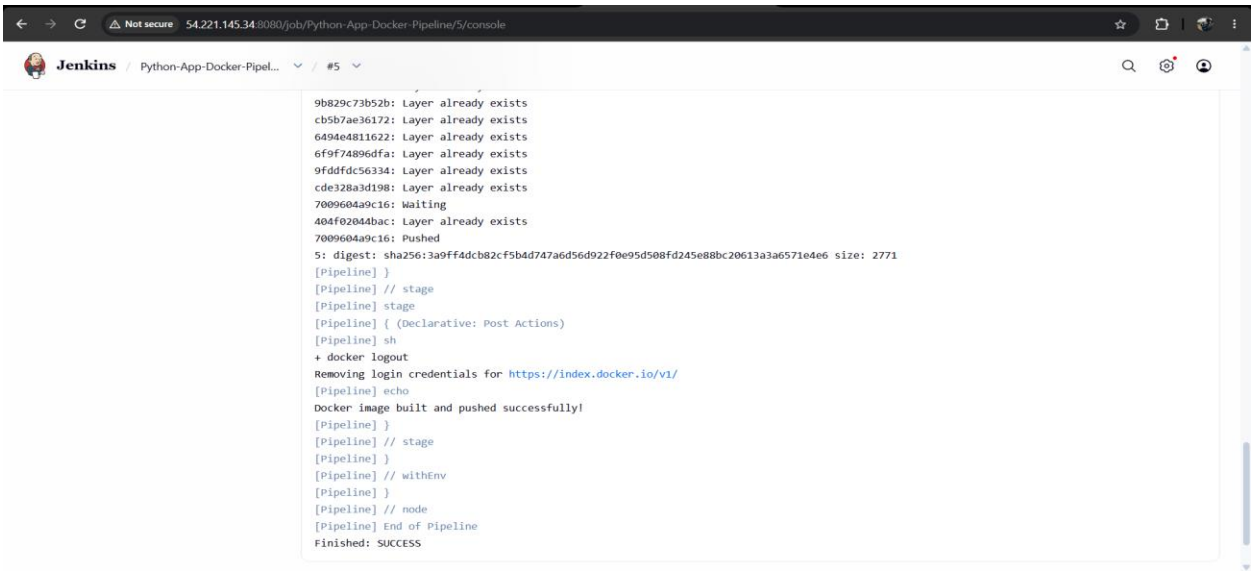
```
    ]]
)
```

```
sh "docker build -t asifshaik6558/python-app:${BUILD_NUMBER} ."
```

The pipeline also used Jenkins credentials to authenticate and push the image.






## Observation

- Jenkins executed the pipeline successfully.
- The Python application source code was checked out from GitHub.
- Docker image build and Docker Hub push completed successfully.
- Build #5 pushed the image tagged `asifshaik6558/python-app:5`.

## Conclusion

Successfully automated the Docker image build and push process using Jenkins Pipeline, GitHub, Docker, and Docker Hub.

# Frontend Application — Dockerize and Push

## Objective

To create a Docker image for a frontend application, run it inside an Nginx container, verify its functionality, and push the image to Docker Hub.

## Steps Used

1. Cloned the assignment repository:

```
git clone https://github.com/betawins/docker-tasks.git
cd docker-tasks/Frontend_based_source
```

2. Inspected the frontend source files:

- `index.html`
- `javascript.js`
- `style.css`
- `todayDeal.js`

3. Created the Dockerfile:

```
FROM nginx:alpine

COPY index.html javascript.js style.css todayDeal.js
/usr/share/nginx/html/

EXPOSE 80
```

4. Built the Docker image:

```
sudo docker build -t asifshaik6558/frontend-app:v1 .
```

5. Started the container:

```
sudo docker run -d \
  --name frontend-app \
  -p 9001:80 \
  asifshaik6558/frontend-app:v1
```

6. Verified the application:

```
sudo docker ps
curl -I http://localhost:9001
```

7. Pushed the image to Docker Hub:

```
sudo docker push asifshaik6558/frontend-app:v1
```

```
ubuntu@ip-172-31-29-148:~$ cd ~
ubuntu@ip-172-31-29-148:~$ git clone https://github.com/betawins/docker-tasks.git
Cloning into 'docker-tasks'...
remote: Enumerating objects: 146, done.
remote: Counting objects: 100% (146/146), done.
remote: Compressing objects: 100% (123/123), done.
remote: Total 146 (delta 12), reused 141 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (146/146), 22.11 MiB | 80.27 MiB/s, done.
Resolving deltas: 100% (12/12), done.
ubuntu@ip-172-31-29-148:~$ cd docker-tasks
ubuntu@ip-172-31-29-148:~/docker-tasks$ ls -la
total 28
drwxrwxr-x  6 ubuntu ubuntu 4096 Oct 16 07:04 .
drwxr-x--- 13 ubuntu ubuntu 4096 Oct 16 07:04 ..
drwxrwxr-x  7 ubuntu ubuntu 4096 Oct 16 07:04 .git
drwxrwxr-x  2 ubuntu ubuntu 4096 Oct 16 07:04 Frontend_based_source
drwxrwxr-x  3 ubuntu ubuntu 4096 Oct 16 07:04 Java_based_source
drwxrwxr-x  4 ubuntu ubuntu 4096 Oct 16 07:04 NodeJs_based_source
-rw-rw-r--  1 ubuntu ubuntu  488 Oct 16 07:04 README.md
ubuntu@ip-172-31-29-148:~/docker-tasks$ cd ~/docker-tasks/Frontend_based_source
ubuntu@ip-172-31-29-148:~/docker-tasks/Frontend_based_source$ ls -laclient_loop: send disconnect: Connection reset by peer
```

```
ubuntu@ip-172-31-29-148: ~/docker-tasks/Frontend_based_source
ubuntu@ip-172-31-29-148:~$ cd ~/docker-tasks/Frontend_based_source
ubuntu@ip-172-31-29-148:~/docker-tasks/Frontend_based_source$ ls -la
total 52
drwxrwxr-x 2 ubuntu ubuntu  4096 Oct 16 07:04 .
drwxrwxr-x 6 ubuntu ubuntu  4096 Oct 16 07:04 ..
-rw-rw-r-- 1 ubuntu ubuntu 22657 Oct 16 07:04 index.html
-rw-rw-r-- 1 ubuntu ubuntu  3345 Oct 16 07:04 javascript.js
-rw-rw-r-- 1 ubuntu ubuntu  9173 Oct 16 07:04 style.css
-rw-rw-r-- 1 ubuntu ubuntu  3948 Oct 16 07:04 todayDeal.js
ubuntu@ip-172-31-29-148:~/docker-tasks/Frontend_based_source$ cat > Dockerfile <<'EOF'
FROM nginx:alpine

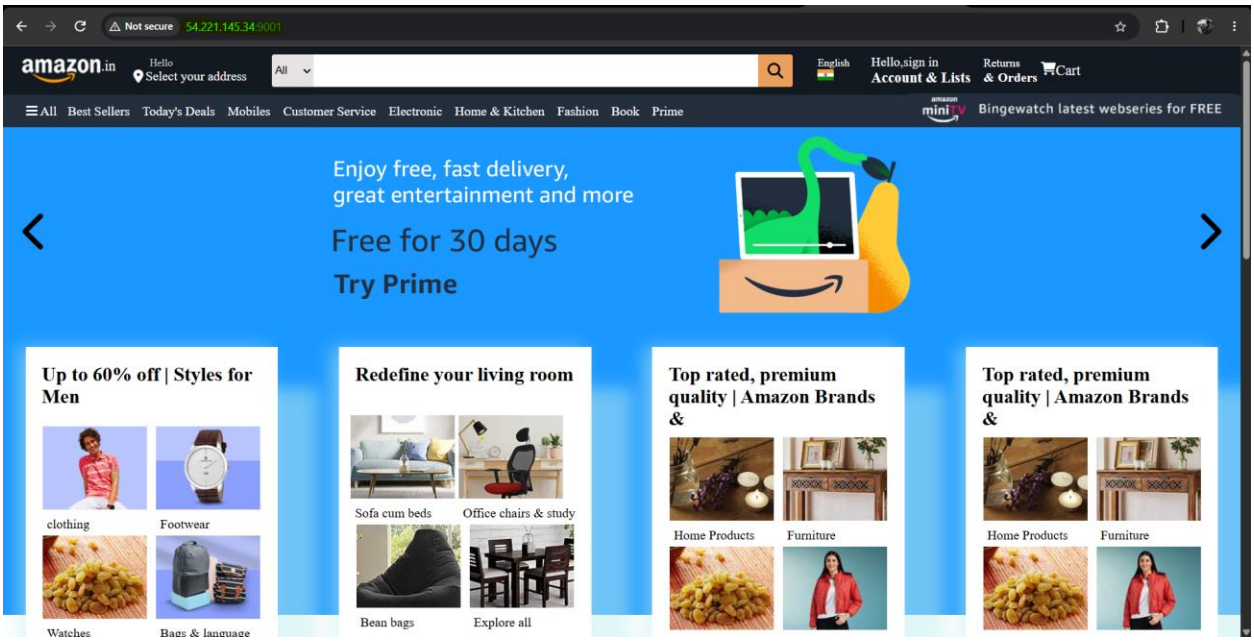
COPY index.html javascript.js style.css todayDeal.js /usr/share/nginx/html/

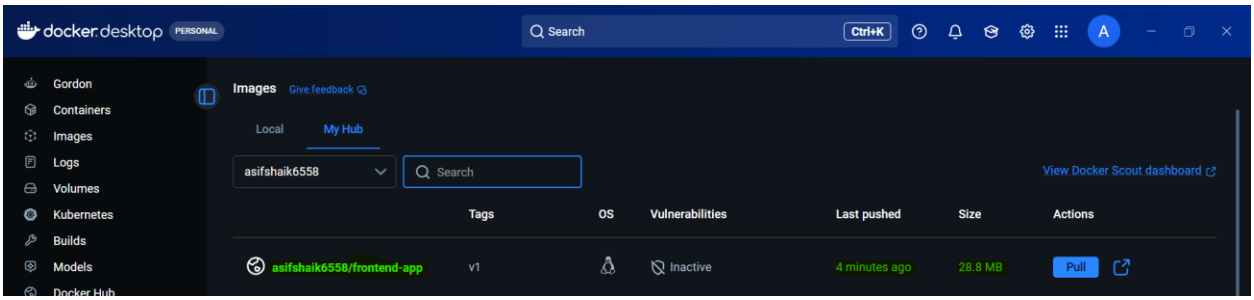
EXPOSE 80
EOF
ubuntu@ip-172-31-29-148:~/docker-tasks/Frontend_based_source$ cat Dockerfile
FROM nginx:alpine

COPY index.html javascript.js style.css todayDeal.js /usr/share/nginx/html/

EXPOSE 80
ubuntu@ip-172-31-29-148:~/docker-tasks/Frontend_based_source$ sudo docker build -t asifshaik6558/frontend-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  44.03kB
Step 1/3 : FROM nginx:alpine
 ---> 72aeb3308ef6
Step 2/3 : COPY index.html javascript.js style.css todayDeal.js /usr/share/nginx/html/
 ---> 9021f70a070d
Step 3/3 : EXPOSE 80
 ---> Running in 15e9fbb83d46
 ---> Removed intermediate container 15e9fbb83d46
 ---> 4c7d9649db10
Successfully built 4c7d9649db10
Successfully tagged asifshaik6558/frontend-app:v1
ubuntu@ip-172-31-29-148:~/docker-tasks/Frontend_based_source$ sudo docker images
                                                
```




## Observation

- The image was built successfully using `nginx:alpine`.
- The frontend container was running on port `9001`.
- The HTTP response returned `200 OK`.
- The image was pushed to Docker Hub successfully.

## Conclusion

Successfully containerized and deployed the frontend application using Nginx and published its Docker image to Docker Hub.

# Java Application — Dockerize and Push

## Objective

To create a Docker image for a Java Spring Boot application using a multi-stage Dockerfile, run the application in a container, verify it, and push the image to Docker Hub.

## Steps Used

1. Navigated to the Java source directory:

```
cd ~/docker-tasks/Java_based_source
```

2. Inspected `pom.xml` and the Java source code. The project used Java 11 and Maven.
3. Created the Dockerfile:

```
FROM maven:3.8.8-eclipse-temurin-11 AS build

WORKDIR /app
COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:11-jre

WORKDIR /app
COPY --from=build /app/target/spring-boot-web.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

4. Built the image:

```
sudo docker build -t asifshaik6558/java-app:v1 .
```

5. Ran the Java application:

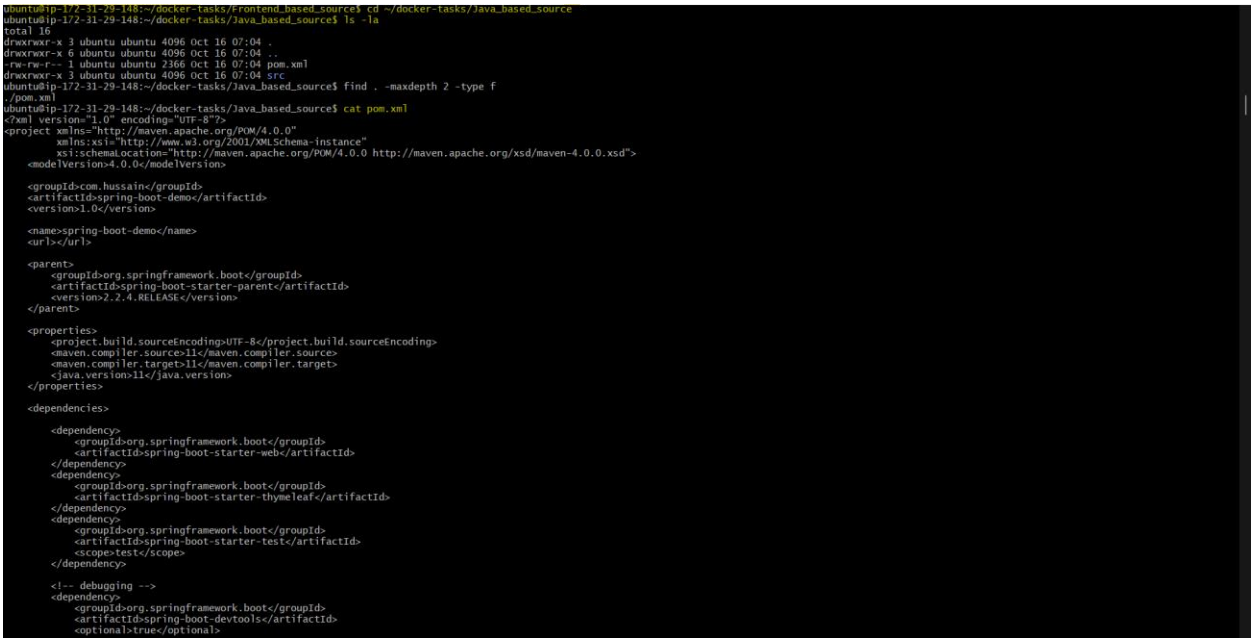
```
sudo docker run -d \
  --name java-app \
  -p 9002:8080 \
  asifshaik6558/java-app:v1
```

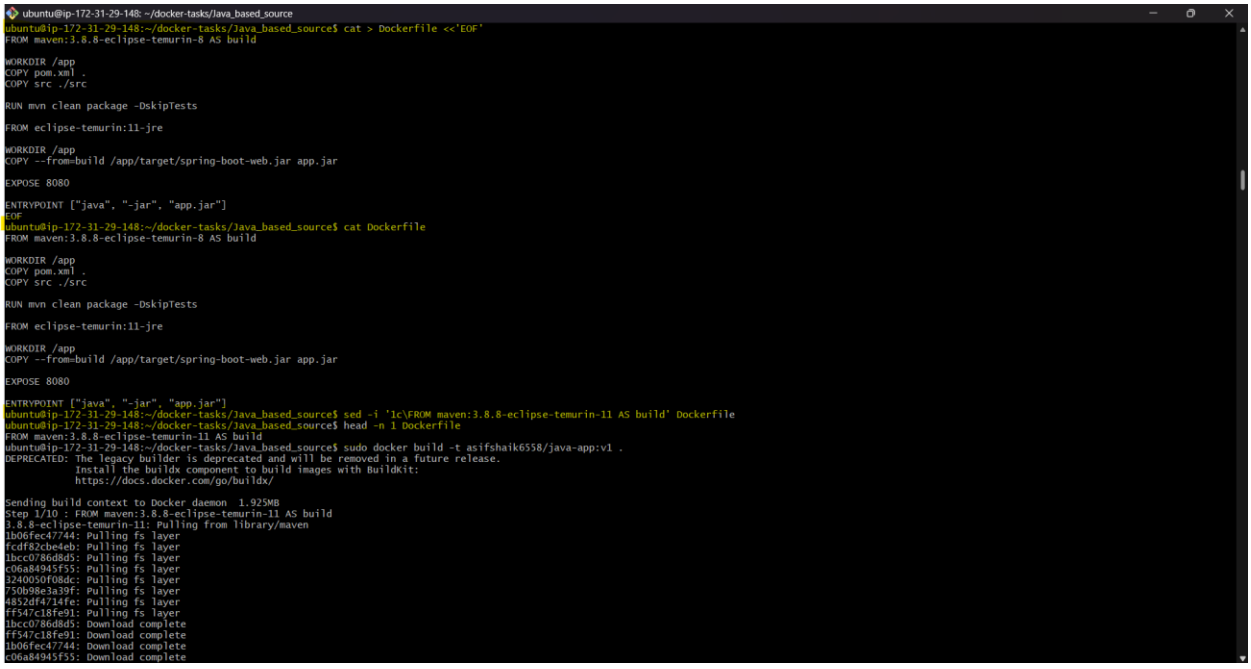
6. Verified the container and application:

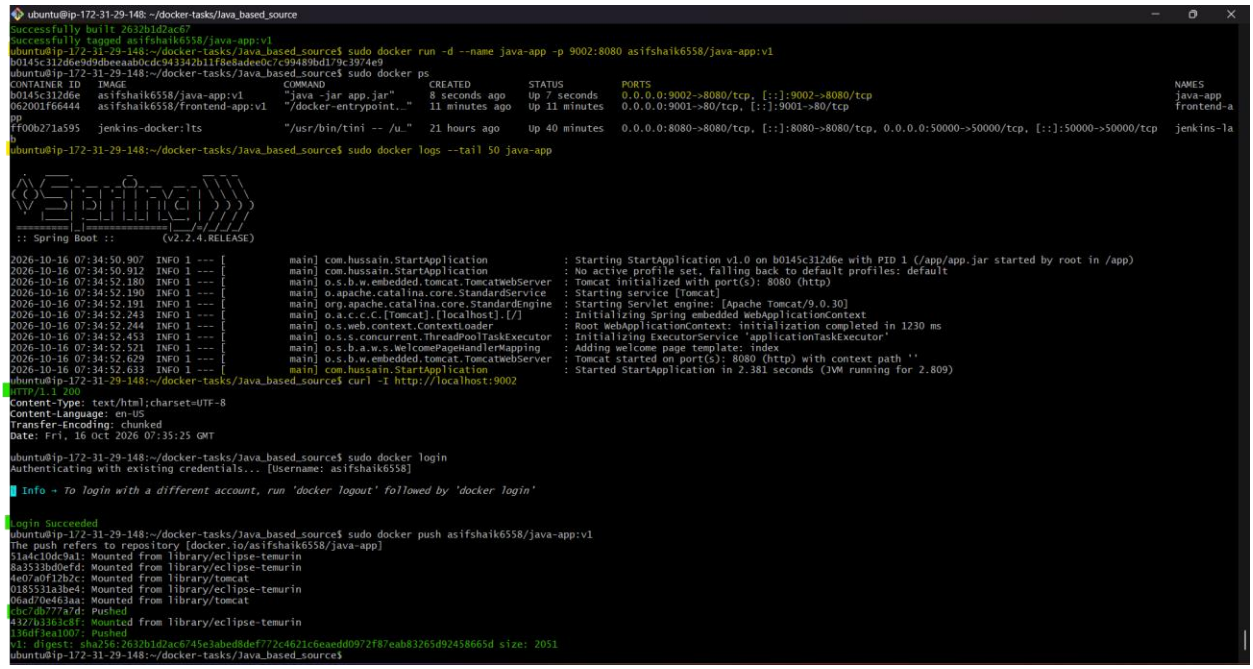
```
sudo docker ps
sudo docker logs java-app
curl -I http://localhost:9002
```

7. Pushed the image:

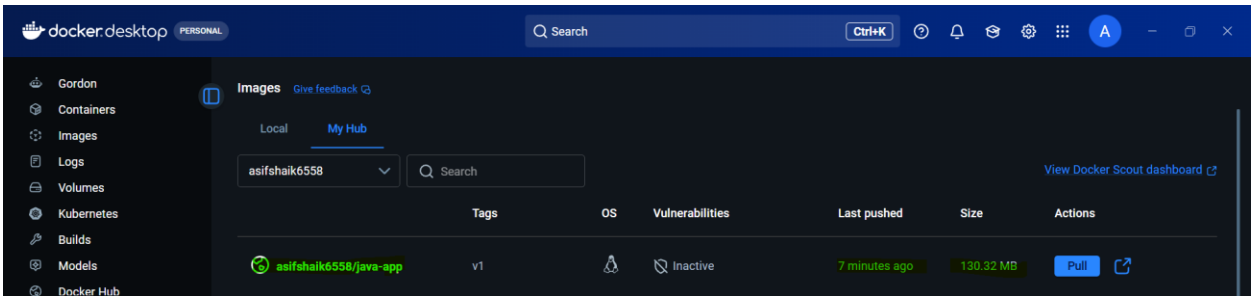
```
sudo docker push asifshaik6558/java-app:v1
```




ubuntu@ip-172-31-29-148: ~/docker-tasks/Java_based_source
Successfully built 2632b1d2ac67
Successfully tagged asifshaik6558/java-app:v1
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$ sudo docker run -d --name java-app -p 9002:8080 asifshaik6558/java-app:v1
b0145c312d6e9d9dbeeaab0cdc9433342b11f6e8adee0c7c99489bd179c3974e9
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
b0145c312d6e asifshaik6558/java-app:v1 "java -jar app.jar" 8 seconds ago Up 7 seconds 0.0.0.0:9002->8080/tcp, [::]:9002->8080/tcp java-app
062001f66444 asifshaik6558/frontend-app:v1 "/docker-entrypoint.…" 11 minutes ago Up 11 minutes 0.0.0.0:9001->80/tcp, [::]:9001->80/tcp frontend-a
pp
ff00b271a595 jenkins-docker:lts "/usr/bin/tini -- /u…" 21 hours ago Up 40 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp jenkins-la
b
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$ sudo docker logs --tail 50 java-app
:: Spring Boot :: (v2.2.4.RELEASE)
2026-10-16 07:34:50.907 INFO 1 --- [ main] com.hussain.StartApplication : Starting StartApplication v1.0 on b0145c312d6e with PID 1 (/app/app.jar started by root in /app)
2026-10-16 07:34:50.912 INFO 1 --- [ main] com.hussain.StartApplication : No active profile set, falling back to default profiles: default
2026-10-16 07:34:52.180 INFO 1 --- [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2026-10-16 07:34:52.190 INFO 1 --- [ main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-10-16 07:34:52.191 INFO 1 --- [ main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.30]
2026-10-16 07:34:52.243 INFO 1 --- [ main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-10-16 07:34:52.244 INFO 1 --- [ main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 1230 ms
2026-10-16 07:34:52.453 INFO 1 --- [ main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2026-10-16 07:34:52.521 INFO 1 --- [ main] o.s.b.a.w.s.WelcomePageHandlerMapping : Adding welcome page template: index
2026-10-16 07:34:52.629 INFO 1 --- [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2026-10-16 07:34:52.633 INFO 1 --- [ main] com.hussain.StartApplication : Started StartApplication in 2.381 seconds (JVM running for 2.809)
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$ curl -I http://localhost:9002
HTTP/1.1 200
Content-Type: text/html;charset=UTF-8
Content-Language: en-US
Transfer-Encoding: chunked
Date: Fri, 16 Oct 2026 07:35:25 GMT
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$ sudo docker login
Authenticating with existing credentials... [Username: asifshaik6558]
Info → To login with a different account, run 'docker logout' followed by 'docker login'
Login Succeeded
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$ sudo docker push asifshaik6558/java-app:v1
The push refers to repository [docker.io/asifshaik6558/java-app]
51a4c10dc9a1: Mounted from library/eclipse-temurin
8a3533bd0efd: Mounted from library/eclipse-temurin
4e07a0f12b2c: Mounted from library/tomcat
0185531a3be4: Mounted from library/eclipse-temurin
06ad70e463aa: Mounted from library/tomcat
cbc7db777a7d: Pushed
4327b1361c8f: Mounted from library/eclipse-temurin
136df3ea1007: Pushed
v1: digest: sha256:2632b1d2ac6745e3abed8def772c4621c6eaedd0972f87eab83265d92458665d size: 2051
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$


Techie Horizon
About
Contact
TECHIE HORIZON
DEVOPS TRAINING INSTITUTE
Tell me
When
Find us on


docker desktop
PERSONAL
Search
Ctrl+K
Gordon
Containers
Images
Logs
Volumes
Kubernetes
Builds
Models
Docker Hub
Images
Give feedback
Local
My Hub
asifshaik6558
Search
View Docker Scout dashboard
Tags
OS
Vulnerabilities
Last pushed
Size
Actions
asifshaik6558/java-app
v1
Inactive
7 minutes ago
130.32 MB
Pull

### Observation

- Maven successfully packaged the Java application.
- The multi-stage build produced a runtime image using Java 11 JRE.
- Spring Boot started successfully, with embedded Tomcat listening on container port `8080`.
- The application returned `200 OK` on host port `9002`.
- The image was pushed to Docker Hub.

### Conclusion

Successfully containerized the Java Spring Boot application using a multi-stage Docker build, deployed it using Docker, verified the HTTP response, and published the image to Docker Hub.

## Node.js/React Application — Build with Tag v1 and Push

### Objective

To create a Docker image for a React application, build its production assets using Node.js, serve them through Nginx, run the container, and push the image to Docker Hub with the required `v1` tag.

### Steps Used

1. Navigated to the Node.js source directory:

```
cd ~/docker-tasks/NodeJs_based_source
```

2. Inspected the React application files, including `package.json`, `package-lock.json`, `public/`, and `src/`.
3. Created the multi-stage Dockerfile:

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci
```

```
COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=build /app/build /usr/share/nginx/html/

EXPOSE 80
```

4. Built the image with the required tag:

```
sudo docker build -t asifshaik6558/nodejs-app:v1 .
```

5. Started the container:

```
sudo docker run -d \
  --name nodejs-app \
  -p 9003:80 \
  asifshaik6558/nodejs-app:v1
```

6. Verified the application:

```
sudo docker ps
curl -I http://localhost:9003
```

7. Pushed the image to Docker Hub:

```
sudo docker push asifshaik6558/nodejs-app:v1
```

```
ubuntu@ip-172-31-29-148: ~/docker-tasks/NodeJs_based_source
ubuntu@ip-172-31-29-148:~/docker-tasks/Java_based_source$ cd ~/docker-tasks/NodeJs_based_source
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ ls -la
total 1228
drwxrwxr-x 4 ubuntu ubuntu    4096 Oct 16 07:04 .
drwxrwxr-x 6 ubuntu ubuntu    4096 Oct 16 07:04 ..
-rw-rw-r-- 1 ubuntu ubuntu 1234987 Oct 16 07:04 package-lock.json
-rw-rw-r-- 1 ubuntu ubuntu     977 Oct 16 07:04 package.json
drwxrwxr-x 2 ubuntu ubuntu    4096 Oct 16 07:04 public
drwxrwxr-x 4 ubuntu ubuntu    4096 Oct 16 07:04 src
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ cat package.json
{
  "name": "zomato_clone",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@emotion/react": "^11.10.5",
    "@emotion/styled": "^11.10.5",
    "@mui/icons-material": "^5.11.0",
    "@mui/material": "^5.11.1",
    "@testing-library/jest-dom": "^5.16.5",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "sass": "^1.57.1",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "eslintConfig": {
    "extends": [
      "react-app",
      "react-app/jest"
    ]
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  }
}
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ find src -maxdepth 3 -type f
src/assets/images/Dine-Out.png
src/assets/images/Online-Food.png
src/assets/images/collection1.webp
src/assets/images/blackLogo.webp
```

```
ubuntu@ip-172-31-29-148: ~/docker-tasks/NodeJs_based_source
src/index.js
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ cat > Dockerfile <<'EOF'
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci

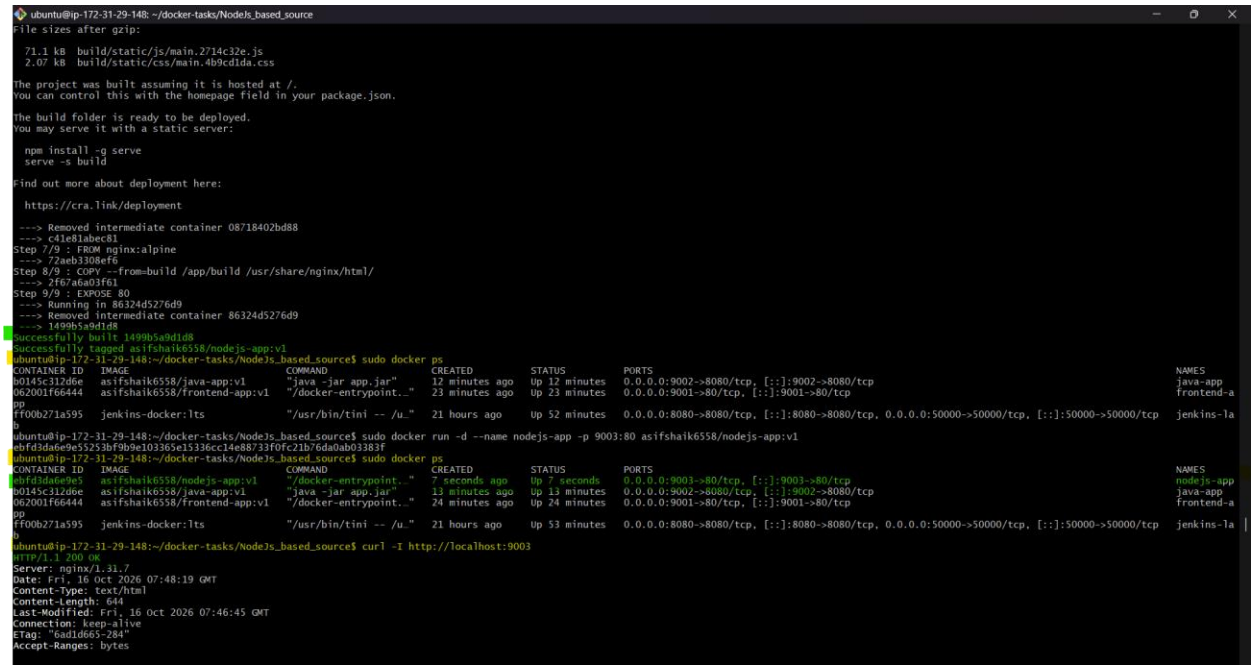
COPY . .
RUN npm run build

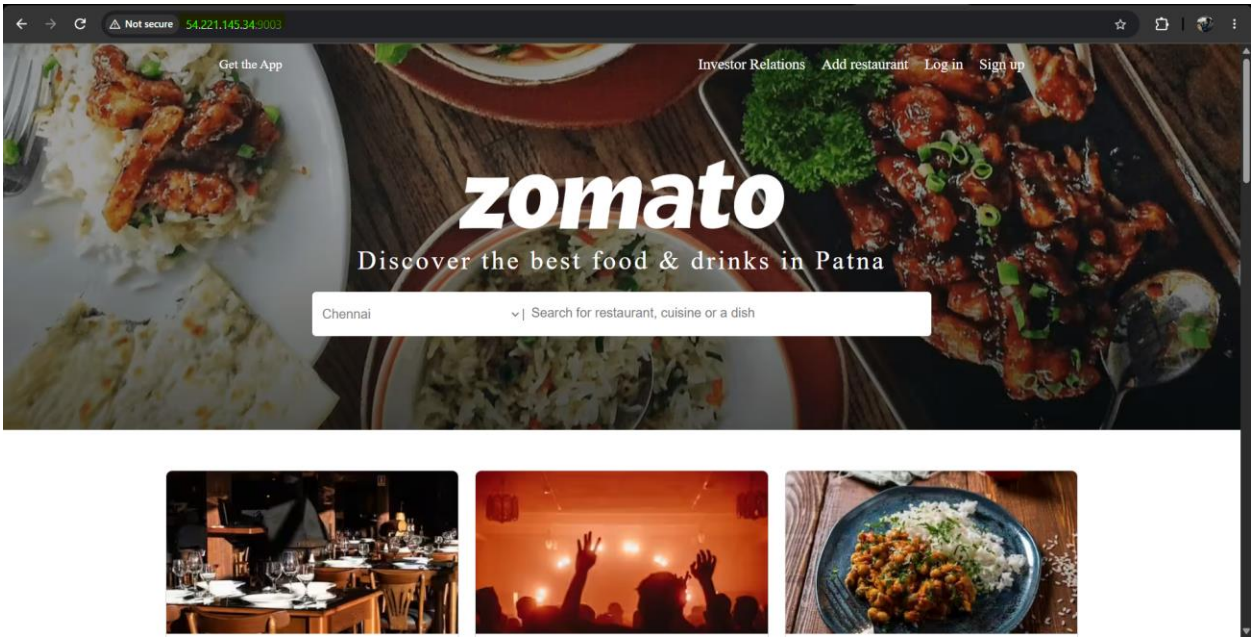
FROM nginx:alpine

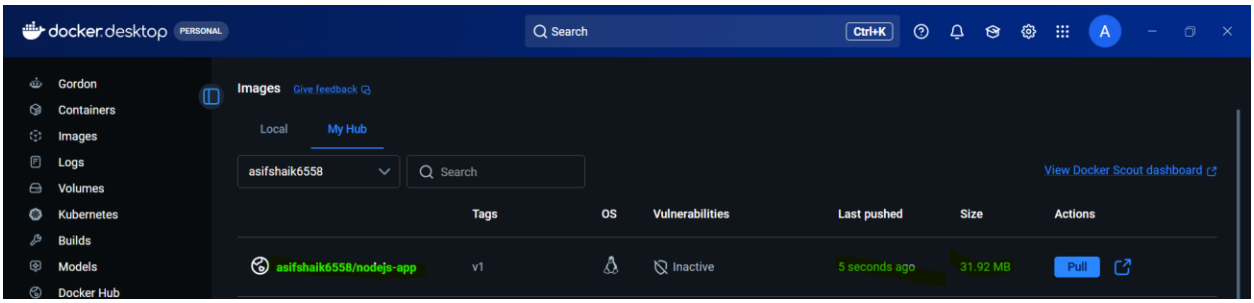
COPY --from=build /app/build /usr/share/nginx/html/

EXPOSE 80
EOF
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ sudo docker build -t asifshaik6558/nodejs-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.17MB
Step 1/9 : FROM node:20-alpine AS build
20-alpine: Pulling from library/node
fff4e2c1b189: Pulling fs layer
6a0ac1617861: Pulling fs layer
4feea04c1543: Pulling fs layer
b2cbbfe903b0: Pulling fs layer
b2cbbfe903b0: Download complete
fff4e2c1b189: Download complete
14f8540414db: Download complete
0f92261bcd1c: Download complete
6a0ac1617861: Download complete
6a0ac1617861: Pull complete
4feea04c1543: Download complete
b2cbbfe903b0: Pull complete
fff4e2c1b189: Pull complete
4feea04c1543: Pull complete
Digest: sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
Status: Downloaded newer image for node:20-alpine
 ---> fb4cd12c85ee
Step 2/9 : WORKDIR /app
 ---> Running in 9c7454f35046
 ---> Removed intermediate container 9c7454f35046
 ---> db08b365b0fc
Step 3/9 : COPY package*.json ./
 ---> 0cdc9efcf6e3
Step 4/9 : RUN npm ci
 ---> Running in f074bc31c1ac
npm warn deprecated w3c-hr-time@1.0.2: Use your platform's native performance.now() and performance.timeOrigin.
npm warn deprecated stable@0.1.8: Modern JS already guarantees Array#sort() is a stable sort, so this library is deprecated. See the compatibility table on MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort#browser_compatibility
npm warn deprecated sourcemap-codec@1.4.8: Please use @jridgewell/sourcemap-codec instead
npm warn deprecated rollup-plugin-terser@7.0.2: This package has been deprecated and is no longer maintained. Please use @rollup/plugin-terser
npm warn deprecated svgo@1.3.2: This SVGO version is no longer supported. Upgrade to v2.x.x.
```


ubuntu@ip-172-31-29-148: ~/docker-tasks/NodeJs_based_source
File sizes after gzip:
71.1 kB build/static/js/main.2714c32e.js
2.07 kB build/static/css/main.4b9cd1da.css
The project was built assuming it is hosted at /.
You can control this with the homepage field in your package.json.
The build folder is ready to be deployed.
You may serve it with a static server:
npm install -g serve
serve -s build
Find out more about deployment here:
https://cra.link/deployment
---> Removed intermediate container 08718402bd88
---> c41e81abec81
Step 7/9 : FROM nginx:alpine
---> 72aeb3308ef6
Step 8/9 : COPY --from=build /app/build /usr/share/nginx/html/
---> 2f67a6a03f61
Step 9/9 : EXPOSE 80
---> Running in 86324d5276d9
---> Removed intermediate container 86324d5276d9
---> 1499b5a9d1d8
Successfully built 1499b5a9d1d8
Successfully tagged asifshaik6558/nodejs-app:v1
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
b0145c312d6e asifshaik6558/java-app:v1 "java -jar app.jar" 12 minutes ago Up 12 minutes 0.0.0.0:9002->8080/tcp, [::]:9002->8080/tcp java-app
062001f66444 asifshaik6558/frontend-app:v1 "/docker-entrypoint.…" 23 minutes ago Up 23 minutes 0.0.0.0:9001->80/tcp, [::]:9001->80/tcp frontend-a
pp
ff00b271a595 jenkins-docker:lts "/usr/bin/tini -- /u…" 21 hours ago Up 52 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp jenkins-la
b
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ sudo docker run -d --name nodejs-app -p 9003:80 asifshaik6558/nodejs-app:v1
ebfd3da6e9e5253bf9b9e103365e15336cc14e88733f0fc21b76da0ab03383f
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ebfd3da6e9e5 asifshaik6558/nodejs-app:v1 "/docker-entrypoint.…" 7 seconds ago Up 7 seconds 0.0.0.0:9003->80/tcp, [::]:9003->80/tcp nodejs-app
b0145c312d6e asifshaik6558/java-app:v1 "java -jar app.jar" 13 minutes ago Up 13 minutes 0.0.0.0:9002->8080/tcp, [::]:9002->8080/tcp java-app
062001f66444 asifshaik6558/frontend-app:v1 "/docker-entrypoint.…" 24 minutes ago Up 24 minutes 0.0.0.0:9001->80/tcp, [::]:9001->80/tcp frontend-a
pp
ff00b271a595 jenkins-docker:lts "/usr/bin/tini -- /u…" 21 hours ago Up 53 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp jenkins-la
b
ubuntu@ip-172-31-29-148:~/docker-tasks/NodeJs_based_source$ curl -I http://localhost:9003
HTTP/1.1 200 OK
Server: nginx/1.31.7
Date: Fri, 16 Oct 2026 07:48:19 GMT
Content-Type: text/html
Content-Length: 644
Last-Modified: Fri, 16 Oct 2026 07:46:45 GMT
Connection: keep-alive
ETag: "6ad1d665-284"
Accept-Ranges: bytes


Not secure 54.221.145.34:9003
Get the App
Investor Relations
Add restaurant
Log in
Sign up
zomato
Discover the best food & drinks in Patna
Chennai
Search for restaurant, cuisine or a dish




## Observation

- The React production build completed successfully.
- The final image used Nginx to serve the generated static files.
- The container was running on port `9003`.
- The HTTP response returned `200 OK`.
- The image was pushed to Docker Hub with the required `v1` tag.

## Conclusion

Successfully containerized the React application using a multi-stage Dockerfile, served its production build through Nginx, verified the application, and pushed the Docker image to Docker Hub.

# WordPress and MySQL Using Docker Compose

## Objective

To deploy WordPress and MySQL together using Docker Compose, configure application-to-database communication, persist data using named volumes, and verify that the WordPress setup page is accessible.

## Steps Used

1. Installed Docker Compose

Initially, the command `sudo docker compose version` failed because the Compose plugin was missing.

Added Docker's official Ubuntu package repository and installed the required packages:

```
sudo apt update
sudo apt install -y docker-compose-plugin
```

Verified the installation:

```
sudo docker compose version
```

Observed version: Docker Compose v5.6.0.

2. Created the project directory

```
mkdir -p ~/wordpress-compose
cd ~/wordpress-compose
```

3. Created `compose.yaml`

The file configured two services: `db` and `wordpress`.

```
services:
  db:
    image: mysql:8.0
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: WpLab_2026_Strong
      MYSQL_ROOT_PASSWORD: RootLab_2026_Strong
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u",
"root", "-pRootLab_2026_Strong"]
      interval: 10s
      timeout: 5s
      retries: 12

  wordpress:
    image: wordpress:latest
    restart: unless-stopped
    depends_on:
      db:
```

```
        condition: service_healthy
    ports:
      - "9004:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_NAME: wordpress
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: WpLab_2026_Strong
    volumes:
      - wordpress_data:/var/www/html

volumes:
  db_data:
  wordpress_data:
```

Note: These were lab credentials. Real deployments should use unique secrets stored securely rather than hardcoding passwords.

4. Validated the configuration

```
sudo docker compose config
```

The configuration was displayed without errors.

5. Started the services

```
sudo docker compose up -d
```

Docker pulled the required images, created the network and named volumes, and started the containers.

6. Verified service status

```
sudo docker compose ps
```

MySQL reported `healthy`, and WordPress was running with port mapping `9004:80`.

7. Tested WordPress

`curl -I` http://localhost:9004

The server returned:

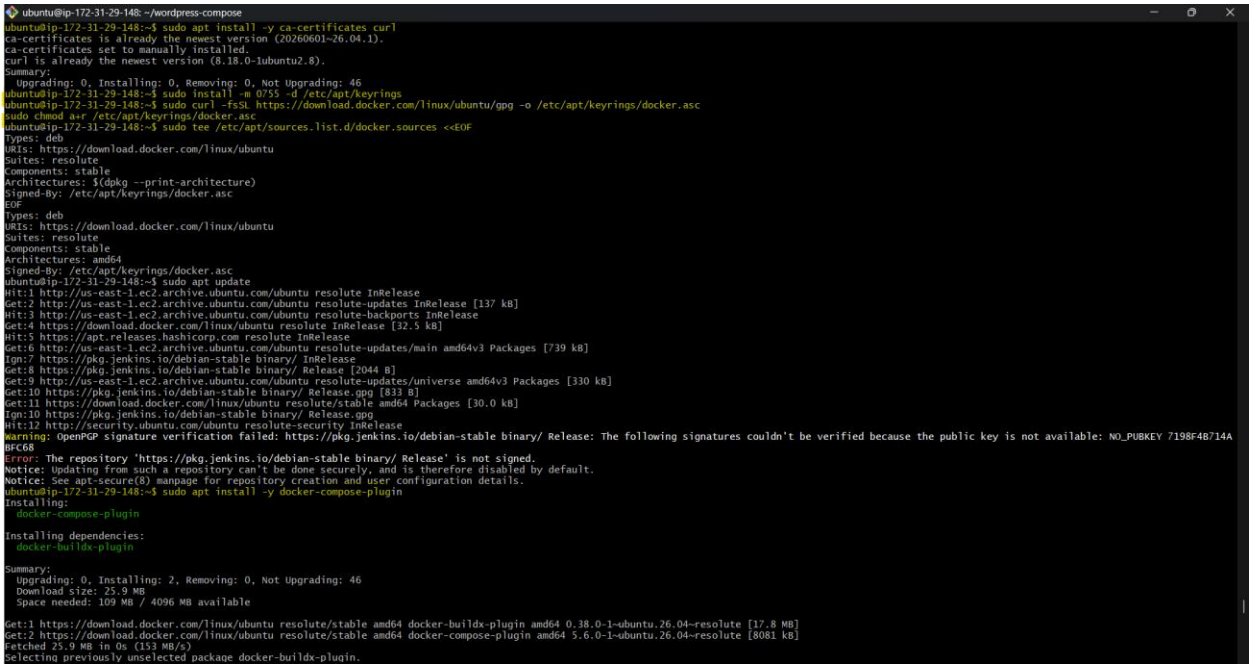
```
HTTP/1.1 302 Found
X-Redirect-By: WordPress
Location: http://localhost:9004/wp-admin/install.php
```

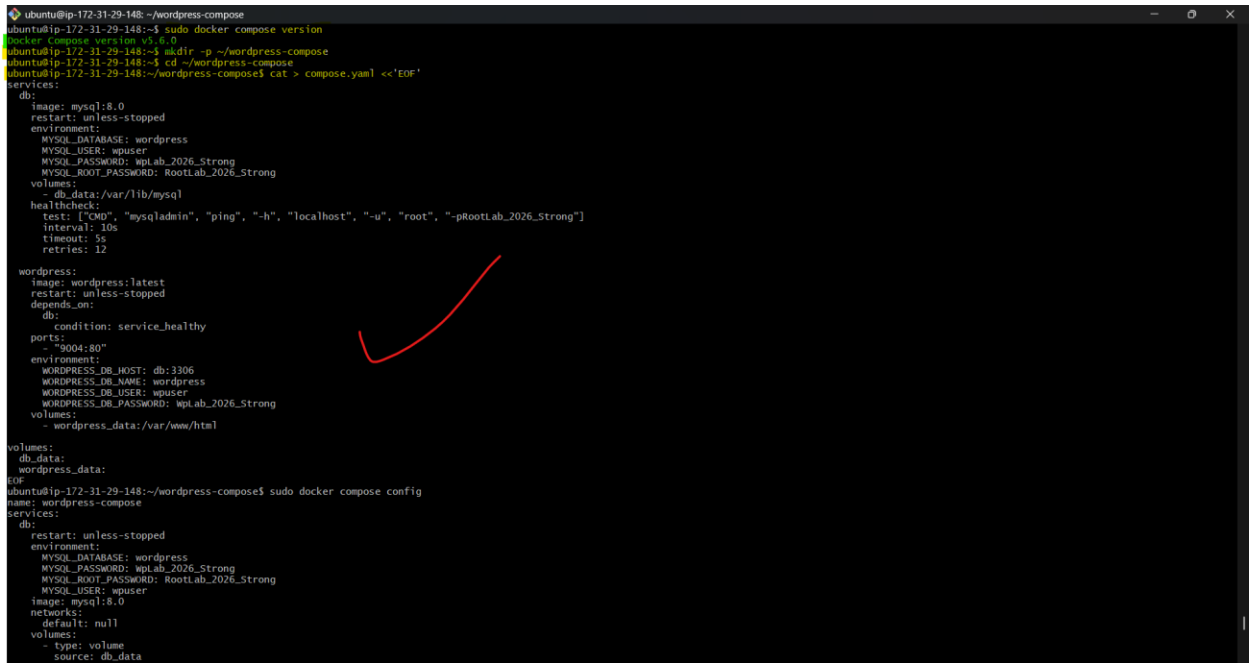
8. Cleaned up the lab

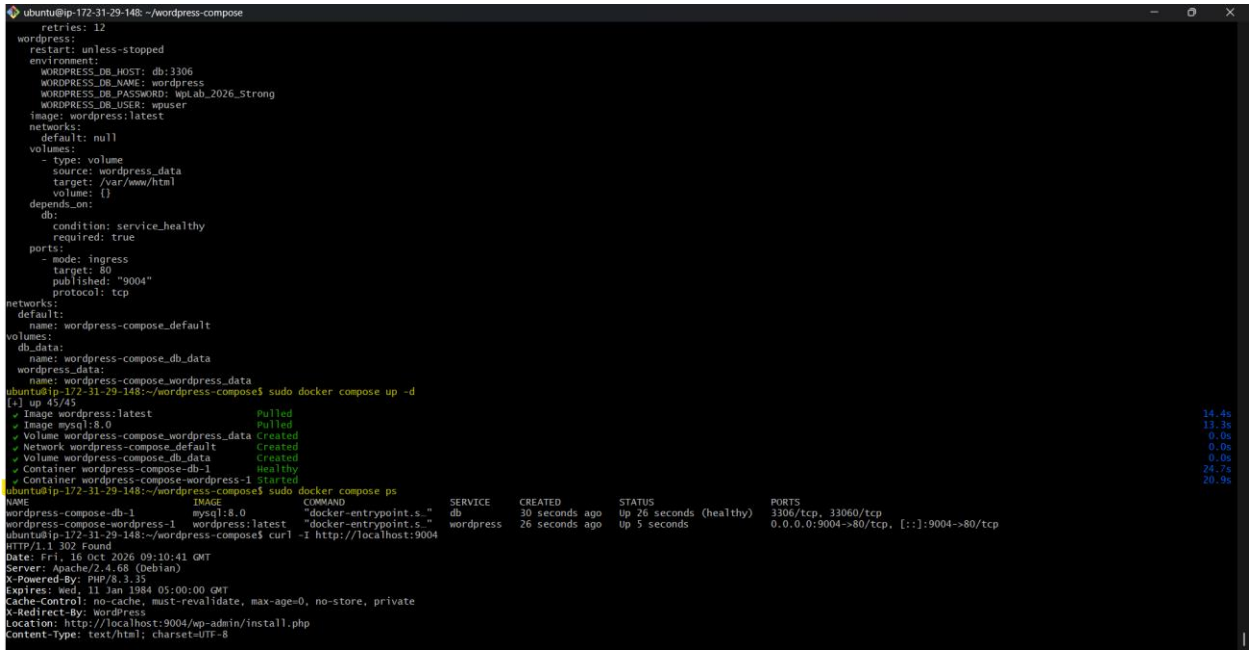
After successful verification, removed the Compose containers, network, and persistent volumes:

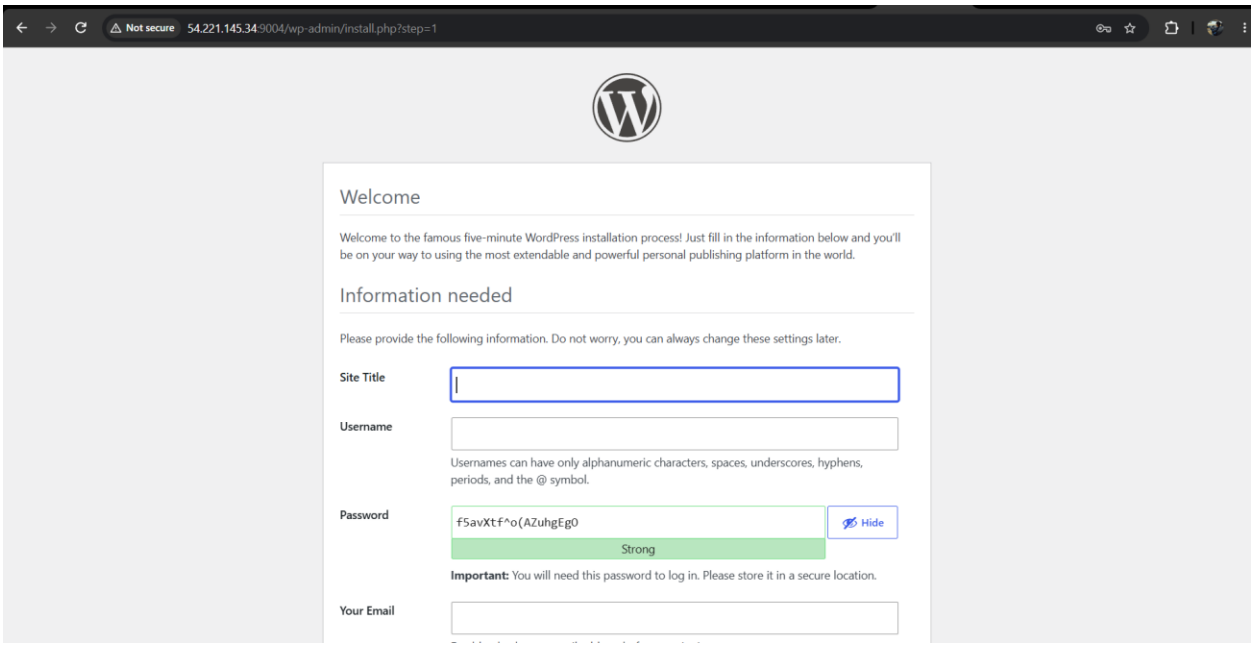
```
sudo docker compose down -v
```

The output confirmed that both containers, both volumes, and the Compose network were removed.




ubuntu@ip-172-31-29-148: ~/wordpress-compose
ubuntu@ip-172-31-29-148:~$ sudo docker compose version
Docker Compose version v5.6.0
ubuntu@ip-172-31-29-148:~$ mkdir -p ~/wordpress-compose
ubuntu@ip-172-31-29-148:~$ cd ~/wordpress-compose
ubuntu@ip-172-31-29-148:~/wordpress-compose$ cat > compose.yaml <<'EOF'
services:
db:
image: mysql:8.0
restart: unless-stopped
environment:
MYSQL_DATABASE: wordpress
MYSQL_USER: wpuser
MYSQL_PASSWORD: WpLab_2026_Strong
MYSQL_ROOT_PASSWORD: RootLab_2026_Strong
volumes:
- db_data:/var/lib/mysql
healthcheck:
test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-pRootLab_2026_Strong"]
interval: 10s
timeout: 5s
retries: 12
wordpress:
image: wordpress:latest
restart: unless-stopped
depends_on:
db:
condition: service_healthy
ports:
- "9004:80"
environment:
WORDPRESS_DB_HOST: db:3306
WORDPRESS_DB_NAME: wordpress
WORDPRESS_DB_USER: wpuser
WORDPRESS_DB_PASSWORD: WpLab_2026_Strong
volumes:
- wordpress_data:/var/www/html
volumes:
db_data:
wordpress_data:
EOF
ubuntu@ip-172-31-29-148:~/wordpress-compose$ sudo docker compose config
name: wordpress-compose
services:
db:
restart: unless-stopped
environment:
MYSQL_DATABASE: wordpress
MYSQL_PASSWORD: WpLab_2026_Strong
MYSQL_ROOT_PASSWORD: RootLab_2026_Strong
MYSQL_USER: wpuser
image: mysql:8.0
networks:
default: null
volumes:
- type: volume
source: db_data


ubuntu@ip-172-31-29-148: ~/wordpress-compose
retries: 12
wordpress:
restart: unless-stopped
environment:
WORDPRESS_DB_HOST: db:3306
WORDPRESS_DB_NAME: wordpress
WORDPRESS_DB_PASSWORD: WpLab_2026_Strong
WORDPRESS_DB_USER: wpuser
image: wordpress:latest
networks:
default: null
volumes:
- type: volume
source: wordpress_data
target: /var/www/html
volume: {}
depends_on:
db:
condition: service_healthy
required: true
ports:
- mode: ingress
target: 80
published: "9004"
protocol: tcp
networks:
default:
name: wordpress-compose_default
volumes:
db_data:
name: wordpress-compose_db_data
wordpress_data:
name: wordpress-compose_wordpress_data
ubuntu@ip-172-31-29-148:~/wordpress-compose$ sudo docker compose up -d
[+] up 45/45
✔ Image wordpress:latest Pulled 14.4s
✔ Image mysql:8.0 Pulled 13.3s
✔ Volume wordpress-compose_wordpress_data Created 0.0s
✔ Network wordpress-compose_default Created 0.0s
✔ Volume wordpress-compose_db_data Created 0.0s
✔ Container wordpress-compose-db-1 Healthy 24.7s
✔ Container wordpress-compose-wordpress-1 Started 20.9s
ubuntu@ip-172-31-29-148:~/wordpress-compose$ sudo docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
wordpress-compose-db-1 mysql:8.0 "docker-entrypoint.s…" db 30 seconds ago Up 26 seconds (healthy) 3306/tcp, 33060/tcp
wordpress-compose-wordpress-1 wordpress:latest "docker-entrypoint.s…" wordpress 26 seconds ago Up 5 seconds 0.0.0.0:9004->80/tcp, [::]:9004->80/tcp
ubuntu@ip-172-31-29-148:~/wordpress-compose$ curl -I http://localhost:9004
HTTP/1.1 302 Found
Date: Fri, 16 Oct 2026 09:10:41 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.3.35
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0, no-store, private
X-Redirect-By: WordPress
Location: http://localhost:9004/wp-admin/install.php
Content-Type: text/html; charset=UTF-8




## Observation

- Docker Compose successfully orchestrated WordPress and MySQL.
- MySQL passed its health check.
- WordPress connected to the database and redirected to its installation page.
- Port `9004` was mapped to the WordPress container's port `80`.
- Named volumes provided persistent storage during the lab.
- The lab resources were removed after testing.

## Conclusion

Successfully deployed and verified a WordPress and MySQL environment using Docker Compose. Confirmed service health, application response, database configuration, and persistent-volume setup before cleaning up the lab.

## Overall Assignment Conclusion

The assignment demonstrated the practical use of Docker for different application types and automated deployment workflows.

- Jenkins: Automated image building and pushing to Docker Hub.
- Frontend: Built and served static frontend files using Nginx.
- Java: Packaged a Spring Boot application using a multi-stage Dockerfile.
- Node.js/React: Built production assets using Node.js and served them through Nginx.

- Docker Compose: Deployed WordPress and MySQL together with service dependencies, health checks, networking, and named volumes.

All five tasks were completed and verified using the command outputs observed during execution.